RESEARCH ARTICLE | SEPTEMBER 21 2023

Developing a novel structured mesh generation method based on deep neural networks

Xinhai Chen (陈新海) ; Jie Liu (刘杰) ; Qingyang Zhang (张庆阳) ; Jianpeng Liu (刘建鹏); Qinglin Wang (王庆林) ; Liang Deng (邓亮); Yufei Pang (庞宇飞)

Physics of Fluids 35, 097137 (2023)
https://doi.org/10.1063/5.0169306

Physics of Fluids ARTICLE pubs.aip.org/aip/pof

Developing a novel structured mesh generation method based on deep neural networks

Cite as: Phys. Fluids 35, 097137 (2023); doi: 10.1063/5.0169306
Submitted: 25 July 2023 · Accepted: 1 September 2023 ·
Published Online: 21 September 2023

Xinhai Chen (陈新海),[1,2] Jie Liu (刘杰),[1,2,a)] Qingyang Zhang (张庆阳),[1,2] Jianpeng Liu (刘建鹏),[3] Qinglin Wang (王庆林),[1,2] Liang Deng (邓亮),[4] and Yufei Pang (庞宇飞)[4]

AFFILIATIONS

[1]College of Computer, National University of Defense Technology, Changsha, China
[2]Laboratory of Digitizing Software for Frontier Equipment, National University of Defense Technology, Changsha, China
[3]College of Science, National University of Defense Technology, Changsha, China
[4]China Aerodynamics Research and Development Center, Mianyang, China

[a)]Author to whom correspondence should be addressed: liujie@nudt.edu.cn

ABSTRACT

In this paper, we develop a novel structured mesh generation method, MeshNet. The core of the proposed method is the introduction of deep neural networks to learn high-quality meshing rules and generate desired meshes. To accomplish this, MeshNet employs a well-designed physics-informed neural network to approximate the potential transformation (mapping) between computational and physical domains. The training process is governed by differential equations, boundary conditions, and *a priori* data derived from coarse mesh generation, which has been disregarded in previous studies. The automatic subdivision of a given domain into quadrilateral elements is achieved through efficient feed-forward neural prediction. A series of experiments are conducted to investigate the robustness of the proposed method. The results across different cases demonstrate that MeshNet is fast and robust. It outperforms state-of-the-art neural network-based generators and produces meshes of comparable or higher quality compared to expensive traditional meshing methods. Furthermore, the proposed method enables fast varisized mesh generation without re-training. The simplicity and computational efficiency of MeshNet make it a novel meshing tool in the discretization part of simulation software.

I. INTRODUCTION

With the development of high-performance computing and physical modeling, numerical simulation has been the centerpiece of the analysis and design processes in the fields of scientific research and engineering technology, such as automotive, aerospace, solid structure, fluid, electromagnetic, biomechanical, and reverse engineering.[1–3] Typically, these simulations require meshes as a basic geometric representation before the physical phenomena can be analyzed with finite element or other numerical methods. The mesh generation process first discretizes the given domain (geometry) into finite mesh elements and then provides an analysis capability to facilitate the numerical solution of partial differential equations (PDEs).[4,5] Starting from the observation that the computational accuracy and efficiency depend very much on the mesh quality, achieving fast and high-quality mesh generation has become one of the primary issues to be considered in numerical simulations.[6–8]

Structured meshes, which comprise entirely of quadrilateral or hexahedral elements, have been widely used in computer-aided design and engineering for many years.[9,10] The advantages of the structured mesh are well understood: (1) since all interior nodes and cells have identical connectivity, structured meshes do not suffer from expensive memory and reduce both the discretization errors and the number of elements as compared to unstructured meshes; (2) on the other hand, the banded non-zero entries possessed by structured meshes can significantly minimize the indirect memory accessing during calculation, thus enabling higher efficiency on modern processors with vector units; (3) meanwhile, structured meshes are eminently suitable for multigrid or other acceleration methods for accurately resolving anisotropic features in some numerical analysis such as boundary layers, shock waves, and highly elastic (or plastic) simulations.[11–13]

Motivated by the aforementioned benefits, extensive research has been made on structured mesh generation over the past four decades.[14] Algebraic methods are often applied directly to structured meshing for simple blocks or tubular structures. Good reviews of some algebraic methods can be found in Refs. 9 and 15. Although these methods are able to achieve optimal efficiency, they are prone to

produce poor-quality or tangling cells in geometric cases with complex boundary shapes. Another attractive structured mesh generation method is the PDE method. This method maps the computational domain to the physical domain by numerically solving a boundary value problem governed by PDE systems.[16] During meshing, the underlying elliptic (or hyperbolic) systems are capable of generating orthogonal and smooth mesh cells due to their natural characteristics, making them widely used for structured meshing in many disciplines. Unfortunately, PDE method-based mesh generation is computationally intensive and time-consuming, especially for large-scale meshing, varisized re-meshing, or real-time analysis scenarios. Thus, there is a great demand for further research and development for a fast and robust structured mesh generation method that balances speed and meshing quality.

In recent years, deep neural network-driven methods have advantages for many applications, and significant progress has been made in knowledge-based physical problems during the last several years.[17–23] Among these, several attempts have been proposed to integrate neural networks into mesh generation tasks. Peng *et al.*[24] developed a hybrid mesh generation method for viscous flow simulations based on an artificial neural network. The method first employs the classical advancing layer method to generate the initial training grids as the training dataset. Then, the introduced artificial neural network is used to predict the advancing direction of the new point. However, the performance of the method is constrained by the initial training mesh, and poor-quality mesh can affect the robustness of the prediction results. Huang *et al.*[25] presented a machine learning-based approach to predict the optimal mesh density for 2D wind tunnel simulations. They first produce over 60 000 simulations (6 TB of data) to establish a dataset and then train a neural network to give inexperienced CFD users a reasonable first guess for mesh refinement. Lu and Tian[26] developed a neural network-based mesh generation program for finite element hydraulic modeling. According to their procedure, the user must manually extract a series of terrain point data as training samples before adopting radial basis function training. Zhang *et al.*[27] proposed a data-driven method to automatically find an appropriate mesh density distribution in 2D domains. Under the supervised learning scheme, they build a training dataset by computing high-accuracy solutions on high-density uniform meshes using a standard FE solver. After that, the network is trained to guide the mesh refinement of local areas. Alexis *et al.*[28] constructed a large-scale training contour dataset using Gmsh and provided a data-driven framework for mesh generation on 2D contours. Chedid and Najjar[29] applied a Kohonen neural network to learn the relationship between mesh density and geometric features of 2D models, during which a training set needs to be carefully prepared with the help of an expert. Overall, the aforementioned methods rely heavily on large-scale training datasets, and developing these supervised learning-based mesh generators requires quite some effort with regard to data creation and processing. In addition, inadequate or noisy data can seriously affect the prediction accuracy of neural networks, which limits their use in practical applications.

The first successful attempt to generate finite element meshes without large-scale datasets was due to Ahmet and Arslan.[30] In their work, boundary points are used as network input to control the generation of inner points. The original complex inner points calculations are replaced with fast feed-forward prediction, and the outputs of neural networks are directly mesh points. However, this method often produces poor-quality mesh cells because it only considers if the newly inserted mesh point belongs to the inner region. Bohn and Feischl[31] investigated the possibility of using deep learning techniques as a black-box mesh refinement tool for PDE problems. Their results show that an optimal mesh refinement strategy can be learned by recurrent neural networks. Chen *et al.*[32] innovatively proposed a novel differential mesh generation method based on unsupervised neural networks. The main insight of their work is employing a physics-informed neural network[33–35] to approximate the transformation between computational and physical domains. Specifically, given a geometry domain, this method first uses a regression model to fit the input boundary curves and provide boundary data for subsequent training. Then, a well-designed neural network is employed to study the potential meshing rules. The governing equations and boundary constraints are embedded in the loss function as penalizing terms during this process. Their results show that the proposed method is capable of producing acceptable meshes on relatively simple geometric cases and achieving high generation efficiency. In order to improve the quality of the generated mesh while mitigating the training data problem, Chen *et al.*[36] modified the penalizing loss terms by introducing an auxiliary line strategy. Initially, their method requires a manual selection of suitable auxiliary lines as ground truth. Thereafter, neural networks are trained to study the underlying meshing rules and generate meshes of the required density. While the auxiliary line strategy offers a way to improve the prediction accuracy of the neural network, this strategy is inherently empirical and introduces extra human intervention. The auxiliary lines cannot be chosen arbitrarily because improper lines can have a negative effect on training convergence.

This paper presents a new neural network-based structured mesh generation method, MeshNet, to overcome this situation and further improve the meshing performance. Specifically, a carefully designed physics-informed neural network is developed to learn the potential transformation (mapping) between computational and physical domains. The high-quality structured meshes are derived based on the non-convex training constrained by governing differential equations, boundary terms, and *a priori* data obtained from coarse mesh, whose information is ignored in the previous studies. The validation of MeshNet is assessed first by comparing the present method with the state-of-the-art neural network-based generators and traditional mesh generation methods on different geometric cases. The experimental results show that with more constrained conditions enforced from the loss terms, MeshNet can produce high-quality meshes in complex geometries, while the existing methods may not guarantee an acceptable mesh without serious mesh distortion or overlapping. Moreover, the experiments on hyperparameters (e.g., coarse mesh sizes, learning rates, and penalty weights) prove that MeshNet is capable of significantly improving the meshing quality without requiring a large-scale dataset beforehand. Finally, we evaluate and compare the meshing overhead of mesh generation methods. The results in all cases demonstrate that MeshNet is fast the robust. It enables high-quality and varisized mesh generation at little cost of feed-forward prediction, making it possible to be used as a novel mesh generator in preprocessing software.

The remainder of the paper is organized as follows. In Sec. II, we introduce the problem definition and provide a detailed description of the proposed method, MeshNet. In Sec. III, we conduct a series of experiments to illustrate the performance and applicability of the

proposed method. Finally, we conclude the paper and discuss the future work in Sec. IV.

II. METHODOLOGY

A. Problem definition

In order to generate a structured mesh for the input geometry, and according to Refs. 9 and 16, we can assume that there exists a potential mapping between the computational and physical domains. Generally, the computational domain (ξ, η) is defined as a simple square domain $[0 \times 1] \times [0 \times 1]$, where every edge is horizontal or vertical. The physical domain (x, y) is the representation of the input geometry. The typical objectives of any structured mesh generation method are to establish a transformation $(\xi, \eta) \rightarrow (x, y)$ between these two domains. The transformation is required to be one-to-one, smooth, and ensure that the generated mesh is consistent with an adequate description of the boundary of the geometry to be discretized.

Algebraic methods and partial differential equation-based methods are the two most common methods used in structured mesh generation.[9,15] The basic principle of the algebraic methods is to continuously interpolate the correspondence specified on the boundary to the interior of the domain and create a mapping of the computational domain to the physical domain by means of an algebraic relation. Despite their simplicity, algebraic interpolation methods usually suffer from mesh distortion problems, i.e., they tend to produce overlapping or degenerate mesh cells, which leads to poor-quality or invalid meshes over complex geometries. Partial differential equation-based methods consider the mesh generation process as a boundary value problem (BVP) under input geometric boundary constraints. The structured mesh is derived by iteratively solving the governing PDE systems, such as elliptic or hyperbolic equation systems. Thompson *et al.*[9] proposed the following Poisson equations to describe the mapping between the physical coordinates and the computational coordinates,

$$\begin{aligned} \frac{\partial^2 x}{\partial \xi^2} + \frac{\partial^2 x}{\partial \eta^2} &= P(\xi, \eta), \\ \frac{\partial^2 y}{\partial \xi^2} + \frac{\partial^2 y}{\partial \eta^2} &= Q(\xi, \eta), \end{aligned} \tag{1}$$

where P and Q are source terms.

The advantage of PDE methods is the ability to smoothly propagate known boundary points to the physical interior, thus generating a valid mesh. They are also insensitive to the boundary singularities and do not generate overlapping meshes if the boundary is not smooth. Unfortunately, PDE methods are cumbersome in cases where fine-grained meshes or large quantities of meshes are required. The expensive meshing overhead has become one of the bottlenecks in current numerical simulation and real-time analysis. Therefore, developing an efficient and robust method that achieves a good balance between speed and meshing quality remains an important challenge and is a field of ongoing research.

B. MeshNet: A novel structured mesh generation method based on deep neural networks

1. General framework

In this work, we propose a new neural network-based structured mesh generation method, MeshNet, to overcome this situation and further improve the meshing performance. Assuming there is a high-quality mapping $\hat{M}$ from the computational domain to the physical domain. We use deep neural networks to learn and approximate this potential mapping in our implementation. The basic philosophy behind the proposed method is to build the neural network M to approximate $\hat{M}$ based on the universal approximation capability of neurons in high-dimensional parameter spaces. Once the network parameters $(\theta_{W,b})$ are determined, the solution of the trained network is able to output the corresponding physical coordinates (x, y) for each computational point (ξ, η), eventually achieving the structured discretization capability in the interior of the given boundaries. This process can be written as follows:

$$\hat{M}(\xi, \eta) \approx M(\theta_{W,b}, \xi, \eta). \tag{2}$$

In our method, the only input data are the input geometry. This enables to avoid quite some effort regarding data creation and eliminates the training dependence on large-scale labeled datasets. For clarity and ease of understanding, the general framework of the proposed method is described in Fig. 1. The procedure to generate the structured mesh is as follows:

Step 1: Fit the input boundaries using regression models.
Step 2: Generate an initial coarse mesh.
Step 3: Construct the loss function using the governing partial differential equation, obtained boundary fitting functions, and coarse mesh data.
Step 4: Build the network structure of MeshNet.

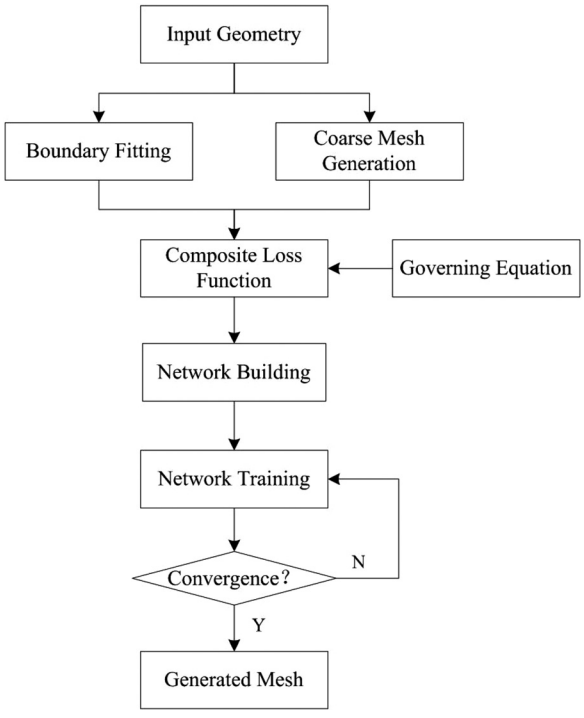

FIG. 1. The general framework of the proposed MeshNet.

Step 5: Minimize the composite loss function constructed in step 3 and update the network parameters using optimization algorithms.
Step 6: Check for convergence. If not, go to step 5.

2. Boundary fitting and coarse mesh generation

Since the geometric boundaries used to characterize the physical domain are often represented by a finite number of discrete points, it is necessary to construct a continuous representation of each physical boundary to obtain sufficient boundary constraints. Thus, the first step of MeshNet involves fitting the input geometric boundaries by means of regression models. The fitting functions derived from the regression model are employed as loss terms to constrain and guide the subsequent neural network training, thus ensuring the generation of a smooth mesh in the interior of the geometry.

To this end, we compared the performance of regression models in the *sklearn* library,[37,38] including polynomial regression, decision tree regression, and support vector machine regression, in terms of their abilities on geometric boundary fitting. We used the least squares metric to evaluate the fitting error of the different models. The results suggest that decision tree regression exhibits the best fitting performance in different geometric cases, thereby being able to minimize error propagation during the training process.

The coarse mesh generation step in the proposed method aims at providing observations for error calibration in each training epoch. Prior works acknowledge something that mesh engineers inherently know, that the simulation accuracy is very sensitive to the underlying mesh quality, where subtle mesh distortions or very few tangling cells may cause instability or non-convergence of the computation. Introducing a small number of observations during the training process contributes to improving the robustness of the neural network model. These observations can be used as measured data to correct the prediction error of the network model at fixed points and subsequently optimize the direction of gradient descent. However, existing neural network-based generators tend to ignore observations. For example, MGNet only employs the governing equation and fitting functions as loss terms to constrain the network training, which leads to problems such as local degenerate mesh and overlapped cells (see the visualization results in Sec. III B). In contrast to the MGNet, we feed the mesh points obtained from coarse mesh into MeshNet. During training, MeshNet measures the predicted coordinates on these mesh points (observations) and embeds their errors into the loss function to determine the optimization direction. For convenience, this step is performed using the automatic differential meshing method. Since the coarse mesh size used in MeshNet is small, this step is fast, usually taking less than 0.1 s. The discussions about the coarse mesh sizes are given in Sec. III D.

3. Composite loss function

In MeshNet, we use the following elliptic partial differential equation system as basic governing equations:

$$\begin{aligned} g_1 x_{\xi\xi} - 2g_2 x_{\xi\eta} + g_3 x_{\eta\eta} &= 0, \\ g_1 y_{\xi\xi} - 2g_2 y_{\xi\eta} + g_3 y_{\eta\eta} &= 0, \end{aligned} \tag{3}$$

where

$$\begin{aligned} &g_1 = x_\eta^2 + y_\eta^2, \quad g_2 = x_\xi x_\eta + y_\xi y_\eta, \quad g_3 = x_\xi^2 + y_\xi^2, \\ &x = x(\xi,\eta), \quad y = y(\xi,\eta), \quad (x,y) \in \Omega, \quad 0 \le \xi,\eta \le 1. \end{aligned} \tag{4}$$

Note that the aforementioned governing equation is replaceable. Other forms of equations, such as hyperbolic equations or their variants, can also be used as governing equations to construct the loss function. According to Eqs. (3) and (4), the composite loss function used in MeshNet is formulated as

$$\begin{aligned} Loss &= MSE_g + \alpha MSE_b + \beta MSE_o \\ MSE_g &= \sum_{i=1}^{N_g} (|g_1 x_{\xi_i\xi_i} - 2g_2 x_{\xi_i\eta_i} + g_3 x_{\eta_i\eta_i}|^2 \\ &\quad + |g_1 y_{\xi_i\xi_i} - 2g_2 y_{\xi_i\eta_i} + g_3 y_{\eta_i\eta_i}|^2), \\ MSE_b &= \sum_{i=1}^{N_b} \left(|x_i - f_{x_i}|^2 + |y_i - f_{y_i}|^2 \right), \\ MSE_o &= \sum_{i=1}^{N_b} \left(|x_i - x_{o_i}|^2 + |y_i - y_{o_i}|^2 \right). \end{aligned} \tag{5}$$

Here, N_g, N_b, and N_o represent the number of points, f is the boundary fitting function, and x_o and y_o are observations obtained from the coarse mesh. MSE_g represents the residual of the governing equations, MSE_b is the sum of the residuals of four boundary fitting functions, and MSE_o is the prediction error on observations obtained from the coarse mesh. $\alpha = (\alpha_{left}, \alpha_{right}, \alpha_{up}, \alpha_{bottom})$ and β are penalty weights employed to balance the contribution of three loss terms. Specifically, we use a dynamic weighting strategy to update the weights α. This strategy utilizes gradient analysis to adaptively assign appropriate weights to the residual terms and provides an automatic way to balance the interplay between MSE_g and MSE_b during network training. The weights for each boundary are computed as

$$\begin{aligned} \alpha_{left} &= \max\left(\frac{\partial MSE_g}{\partial \theta_{W,b}}\right) \Big/ \operatorname{mean}\left(\frac{\partial MSE_{b\text{-}left}}{\partial \theta_{W,b}}\right), \\ \alpha_{right} &= \max\left(\frac{\partial MSE_g}{\partial \theta_{W,b}}\right) \Big/ \operatorname{mean}\left(\frac{\partial MSE_{b\text{-}right}}{\partial \theta_{W,b}}\right), \\ \alpha_{up} &= \max\left(\frac{\partial MSE_g}{\partial \theta_{W,b}}\right) \Big/ \operatorname{mean}\left(\frac{\partial MSE_{b\text{-}up}}{\partial \theta_{W,b}}\right), \\ \alpha_{bottom} &= \max\left(\frac{\partial MSE_g}{\partial \theta_{W,b}}\right) \Big/ \operatorname{mean}\left(\frac{\partial MSE_{b\text{-}bottom}}{\partial \theta_{W,b}}\right), \end{aligned} \tag{6}$$

where $MSE_{b\text{-}left}$, $MSE_{b\text{-}right}$, $MSE_{b\text{-}up}$, and $MSE_{b\text{-}bottom}$ denote the residuals on each boundary. As for β, we choose to use a static weighting strategy to enforce a fixed penalty weight for introduced observations in MeshNet. The effect of penalty weights on meshing performance is discussed in Sec. III D.

In theory, when the governing equation and boundary terms are satisfied during the convergence process (MSE_g and MSE_b approach to zero), the constructed neural network is able to automatically learn high-quality meshing rules from the high-dimensional space, resulting in a neural network-based mesh generator. However, in practice, it is not always guaranteed that the desired result can be obtained within a limited training overhead, especially for complex geometry cases.

To solve this problem, MeshNet constructs a combined loss function by introducing an observation term and multiple penalty weights. Among them, the introduced observation term can provide *a priori* knowledge for training. The satisfaction of the observation term can, in turn, enforce the other loss terms, thus accelerating the convergence of training. Multiple penalty weights can help achieve a better balance between different loss terms and further improve the robustness of the proposed method.

4. Network building and training

Once the composite loss function is available, the next step consists in building the neural network. Here, instead of employing a very deep network structure, we introduce a lightweight neural network model to learn the underlying mapping from the computational domain to the physical domain. The detailed architecture is shown in Fig. 2.

The input to the network is a series of point coordinates $\{(\xi_i, \eta_i)_{i=1}^{i=N}\}$ randomly sampled within the computational domain (both interior and boundary). Before feeding the input coordinates into the hidden layers, we first introduce an augmentation layer to expand the original input coordinates to a higher dimension. This layer uses four linear operators to expand the input two-dimensional feature space to eight dimensions. This augmentation is defined as follows:

$$\{(\xi,\eta)\}\mapsto\{pow(\xi,\eta),\ \sin(\xi,\eta),\ \cos(\xi,\eta),(\xi,\eta)\}, \tag{7}$$

where $pow(\cdot)$, $\sin(\cdot)$, and $\cos(\cdot)$ compute the square, sine, and cosine of the input coordinates (ξ, η), respectively. After the augmentation, fully connected hidden layers are employed to learn the potential high-quality meshing rules. In MeshNet, we build a network structure consisting of four hidden layers with 50 neurons per layer. This lightweight structure can be easily deployed on modern processors (even regular CPUs) for network training. The four hidden layers exploit the nonlinear learning ability of neurons in high-dimensional space to approximate the mapping from the computational domain to the physical domain. The output of the network is directly the physical coordinates (x, y) corresponding to the input.

Since there typically exists excessive feature information in hidden layers (known as information overload), it is desirable to filter extraneous information and focus learning attention on critical information that contributes to meshing instead of distributing the same weights to each feature channel. Obviously, this task requires an attention mechanism, and it turns out that the adjustment of irregular channel weights is crucial to find high-quality solutions. In MeshNet, we embed two fully connected layers in hidden layers as attention modules. This module design is inspired by the soft attention technique widely used in recurrent neural networks.[39,40] The mathematical expression of the *k-th* hidden layer for the residual neural network is written as

$$Y^k = \sigma(Y^{k-1}W^{k-1} + b^{k-1}) \otimes Y^{k-1}, \tag{8}$$

where Y represents the output of each hidden layer, σ represents the activation function, and $\otimes$ denotes the point multiplication operation. Eventually, by using automatic differentiation techniques, the network output and its derivatives are used to compute the composite loss function. "1" represents that the outputs x and y do not need to be differentiated. Instead, they are output directly and remain constant throughout the optimization process.

During training and optimization, the basic idea of MeshNet is to find the optimal network parameters θ that minimize the loss function defined in Eq. (5). In our implementation, both the Adam and L-BFGS-B optimizer[41] are employed to train the MeshNet. We first apply the improved gradient descent optimizer, Adam, to update the

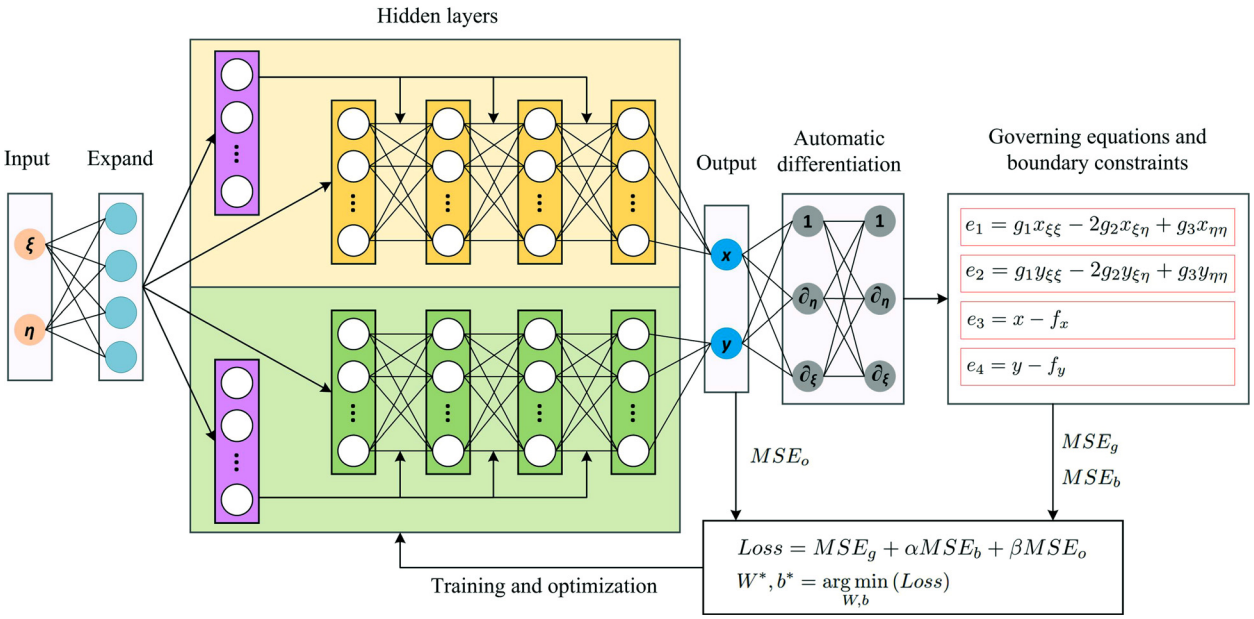

FIG. 2. The detailed architecture of the proposed MeshNet.

network parameters θ. The gradient descent training process can be written as

$$\theta_{W,b} = \theta_{W,b} - \mu\left(\frac{\partial Loss}{\partial \theta_{W,b}}\right), \tag{9}$$

$$W^*, b^* = \arg\min_{W,b}(Loss), \tag{10}$$

where μ denotes the learning rate. We set the initial learning rate to 1×10^{-3} and decay 0.9 every 1000 epochs. The training process of the MeshNet model is unsupervised, meaning that the proposed MeshNet does not require any labeled datasets to provide *a priori* knowledge. Instead, the input to MeshNet consists of mesh points sampled from the interior of the domain, boundaries, and coarse grids. These point data are fed into the combined loss function to compute the corresponding loss terms. Specifically, MSE_g takes input of the data points sampled inside the computational domain. MSE_b computes the mean squared error corresponding to the boundary conditions, and MSE_o refers to the prediction error at the points generated in coarse mesh generation step. The ratio of interior points to boundary points is 4:1, and the number of coarse mesh points is controlled by the parameter N_d. The total epoch of Adam training is set to 5000, and the mini-batch size is set to 128. After that, we apply the Quasi-Newton optimizer, L-BFGS-B, to fine-tune the network. The advantages of this optimizer include fast convergence and low memory overhead. We set the full-batch size to 1000 and the stop criterion of this optimizer to the minimum *float* value that can be reached. After suitable training, the user assigns a mesh size, and the proposed MeshNet enables high-quality and varisized mesh generation at a little cost of feed-forward prediction. The simplicity and computation speed of MeshNet make it possible to be used as a novel mesh generator in preprocessing units.

III. RESULTS AND DISCUSSION

A. Experimental setup

In this section, we conduct a series of experiments to evaluate the performance of MeshNet and demonstrate its meshing capabilities. The experiments are divided into four parts. First, we visualize the meshing results of MeshNet on eight geometric cases and compare them with existing neural network-based generators and widely used traditional meshing methods (Sec. III B). Second, we perform experiments to validate the MeshNet design by in-depth loss convergence analysis in Sec. III C. Third, we explore the effects of various parameter settings, including coarse mesh sizes, penalty weights, learning rates, and batch sizes on the meshing performance of MeshNet (Sec. III D). Finally, we compare the meshing overhead coming from MeshNet with those produced using traditional techniques to verify the engineering application of the proposed method (Sec. III E).

Specifically, the neural network-based generators used for comparison are the MGNet proposed in Ref. 32 and the Auxiliary method in Ref. 36. Traditional methods include algebraic and PDE methods. We use Lagrange Transfinite Interpolation (TFI) for the algebraic method, and the governing equation in the PDE method is given in Eqs. (3) and (4). As for neural network generators, the activation function we use is the *tanh* function. This function introduces nonlinear characteristics into our networks, which is defined as

$$\sigma(Y) = \tanh(Y) = \frac{\sinh Y}{\cosh Y}. \tag{11}$$

Although higher training efficiency can be achieved on the GPUs, we still conducted all experiments on CPUs for comparison purposes. The computing platform we used is Intel Xeon E5-2660 CPU with 2.6 GHz clock speed, which is common in engineering applications. The neural network and automatic differentiation are implemented with TensorFlow 1.14 (Ref. 42) and *tf.gradients*().

B. Visualization results

We start by evaluating and analyzing the meshing results of different structured mesh generation methods. All the methods are validated against eight geometric cases (1–8) to demonstrate the general performance.

One major problem imposed on structured mesh generation has been the ability to provide high-quality meshes. From this aspect, evaluating the quality of the generated mesh is an important part of a meshing algorithm. Figure 3 depicts the meshing results of different methods on geometric case 1. From visualization results, we can see that the transfinite interpolation method tends to produce slivers in the unsmooth near-wall regions, such as those near the discontinuity points of the upper and right boundaries in case 1. These poorly shaped cells can seriously affect the accuracy of the solution or even lead to non-convergence of the simulation.

The MGNet constrained by the control equations can slightly improve the quality of the cells near the boundary, but the overall quality of the mesh still needs to be optimized. As for the PDE method, despite its expensive meshing overhead, this method is able to generate meshes with relatively good quality. Meanwhile, it is clear that methods incorporating *a priori* data (Auxiliary method and MeshNet) allow an accurate approximation of the mapping from the computational domain to the physical domain. Among them, the proposed MeshNet achieves better meshing performance. It exhibits comparable meshing results to the expensive PDE method.

In order to better analyze the meshing performance of different methods, we employ six commonly used quality criteria to quantitatively evaluate the quality of the generated meshes. The six criteria are:

- Area Ratio (AR_{avg}): this criterion is a quality metric for mesh smoothness. It includes the measurement of the area of adjacent cells, which is calculated as follows:

$$max\left[Size_i/minSize_j, maxSize_j/Size_i\right], \tag{12}$$

  where $Size_i$ is the area of the cell i and $minSize_j$ ($maxSize_j$) is the minimum (maximum) area of the adjacent cell j. Here, we use the average area ratio of the entire mesh to evaluate the quality of the underlying mesh.
- Length Ratio x and y (LR_x and LR_y): computes the ratio between the distance from the chosen coordinate direction (x or y) to the $index + 1$ point and the distance to the $index-1$ point. We assume that the larger of the two distances is always divided into the smaller, so the value of this criterion ranges between 0 and 1.
- Aspect Ratio (*Aspect*): aspect ratio for quadrilateral cells computes the ratio of the average length and average width. It is mainly used to determine the quality based on the degree of conformity and to identify degenerate elements. The value of this criterion is always greater than or equal to 1, with a value of 1 representing a square.

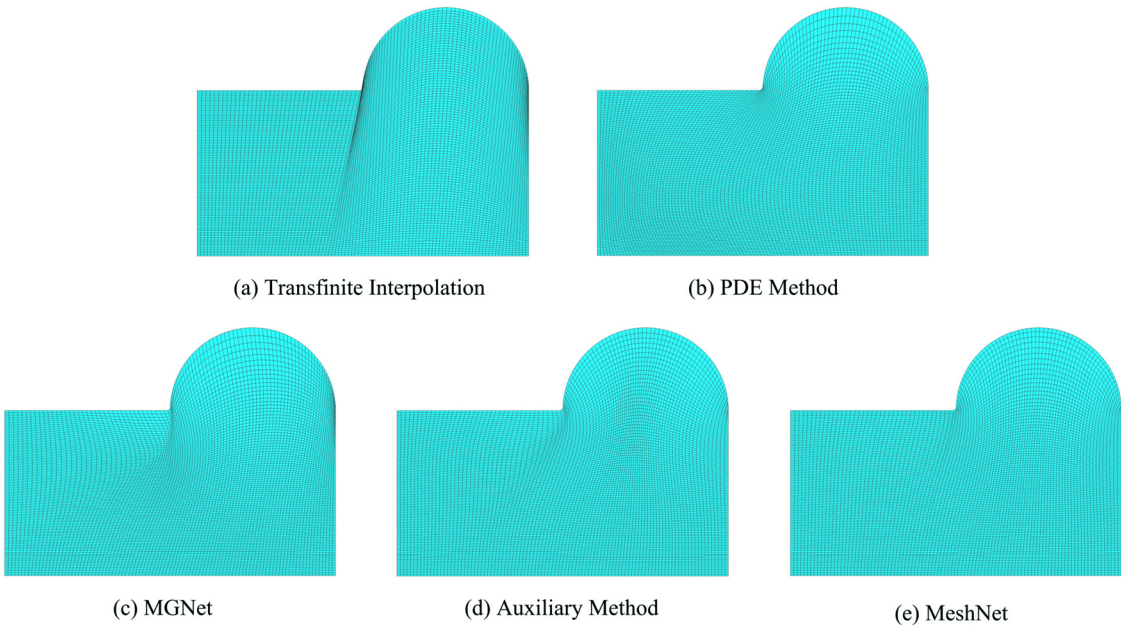

FIG. 3. The meshing results of different methods on geometric case 1.

- Smoothness x and y (S_x and S_y): smoothness criterion is computed from the normalized turning angle of three adjacent points in the chosen coordinate direction (x or y). The value of smoothness varies between 0 and 1, where 1 indicates the three points are colinear and 0 indicates a 180 deg turn, or the mesh is folded onto itself.
- Maximum Included Angle ($Max.A_{avg}$): includes the measurement of cell skewness and distortion. This criterion is represented by the included maximum angle of each mesh cell. Here, we use the average maximum included angle of all mesh cells to evaluate the mesh quality.
- Equiangle Skewness ($Skew$): the equiangle skewness is denoted as the maximum ratio of the cell's included angle to the angle of an equilateral element. It is calculated as

$$\max\left[\frac{(Q_{\max}-Q_e)}{(180-Q_e)},\frac{(Q_e-Q_{\min})}{Q_e}\right], \tag{13}$$

where Q_e is the angle for the equilateral cell and Q_{max} and Q_{min} are the largest and smallest angles in the mesh cell, respectively. A value of 0 indicates an equilateral cell, while 1 indicates a degenerate cell.

Table I summarizes the quality statistics of different mesh generation methods on geometric case 1. The quantitative results prove again that MeshNet is a valid structured mesh generator. The mesh generated by MeshNet is able to achieve good quality evaluation results on different criteria. It gives values of 99.494 and 0.106 for the maximum included angle and equiangle skewness criteria, respectively, which are optimal in the comparison of different meshing methods.

Figure 4 illustrates the meshing results of different methods on geometric case 2. One can easily see that the transfinite interpolation method, MGNet, and the Auxiliary method fail to produce a valid mesh. The meshes generated by these methods suffer from seriously skewed or distorted mesh cells [see Fig. 4(a)] or folded meshes near convex boundaries [see Figs. 4(c) and 4(d)]. However, it shall be noted that the method developed in this work helps in avoiding mesh overlapping and leading to the minimum mesh line distortion.

The corresponding quality statistics of this geometry are shown in Table II. The results show that MeshNet outperforms existing neural network-based mesh generators and yields the best evaluation results on six different criteria. One main reason lies in the stronger learning capability of the proposed method. By building a well-designed neural network, MeshNet is able to accurately learn the

TABLE I. Quality statistics of different mesh generation methods on geometric case 1.

Case 1	AR_{avg}	LR_x	LR_y	$Aspect$	S_x	S_y	$Max.A_{avg}$	$Skew$
TFI	1.218	1.405	1.336	1.870	0.992	1.000	107.631	0.196
PDE method	1.020	1.405	1.371	1.814	0.997	0.997	105.880	0.177
MGNet	1.032	1.534	1.215	1.868	0.996	0.998	104.659	0.164
Auxiliary method	1.041	1.405	1.369	1.904	0.997	0.998	99.990	0.112
MeshNet	1.021	1.405	1.346	1.871	0.997	0.998	99.494	0.106

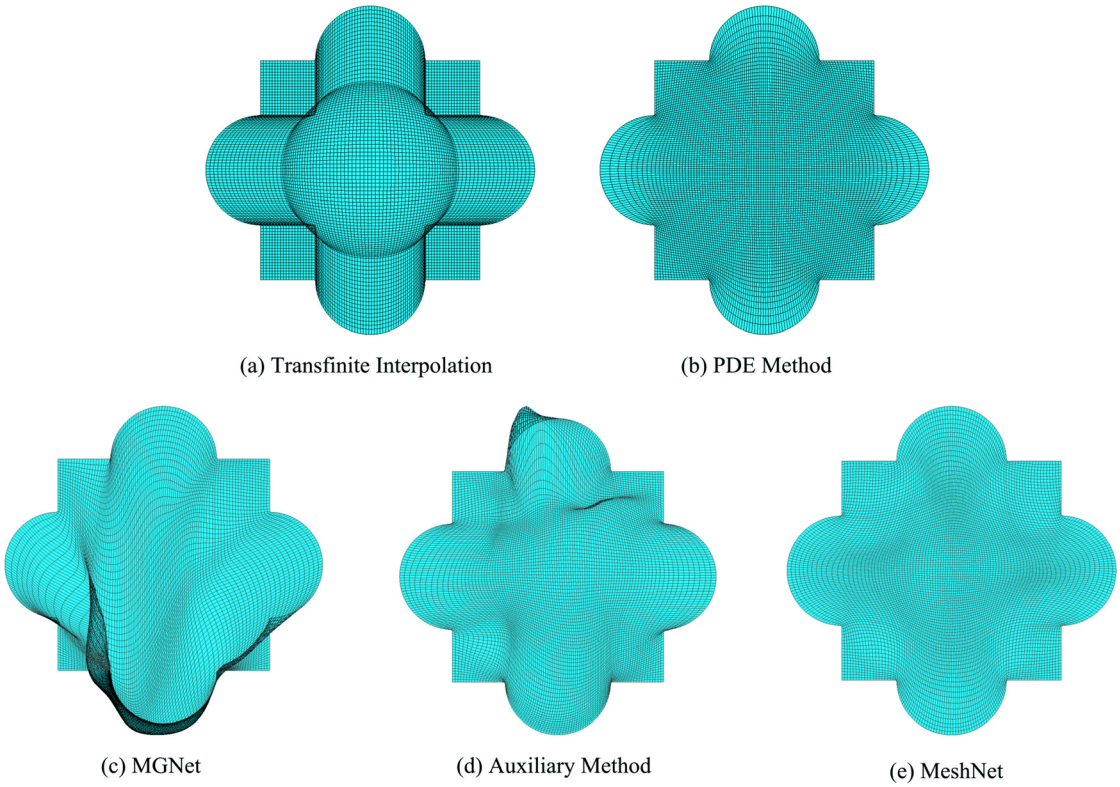

FIG. 4. The meshing results of different methods on geometric case 2.

potential mapping between computational and physical domains from the high-dimensional space, which eventually results in an acceptable mesh. Moreover, benefiting from the introduced composite loss function and weighting strategies, MeshNet is able to make full use of the prior data obtained from the coarse mesh and learn high-quality meshing rules more efficiently. We can also see that, although the Auxiliary method [in Fig. 4(d)] would improve the mesh quality compared to the results produced by MGNet [in Fig. 4(c)], in this case, the points of the mesh still fall outside the domain, and mesh lines intersect within the domain. One reason is that the information provided by the auxiliary lines is limited, and the method is difficult to implement in geometries with irregular boundaries; on the other hand, presetting the auxiliary lines is a tedious task that requires expert knowledge in order to achieve good results. For complex geometric cases, the auxiliary lines must be applied with cautions: they may cause boundary nodes to overtake or collapse to their neighbors. Therefore, an automatic method like MeshNet that extracts *a priori* knowledge directly from a coarse mesh without manual intervention is highly desirable.

Similar results can be seen in the meshing results of geometric case 3. As shown in Fig. 5, the concave boundary layout of the geometric domain has serious mesh distortion and overlapping when using the TFI, MGNet, or Auxiliary methods without the observation term. The improved MeshNet is able to alleviate the aforementioned problem, eliminate the mesh distortion, and achieve comparable meshing performance with the PDE method.

TABLE II. Quality statistics of different mesh generation methods on geometric case 2.

Case 2	AR_{avg}	LR_x	LR_y	*Aspect*	S_x	S_y	$Max.A_{avg}$	*Skew*
TFI	1.742	2.186	2.186	2.279	0.987	0.987	120.260	0.336
PDE method	1.053	1.432	1.432	3.583	0.996	0.996	104.860	0.166
MGNet	2.422	2.071	15.822	384.952	0.980	0.982	132.151	0.470
Auxiliary	1.588	1.810	11.338	22.236	0.990	0.985	116.269	0.294
MeshNet	1.076	1.700	1.446	3.658	0.991	0.992	110.892	0.234

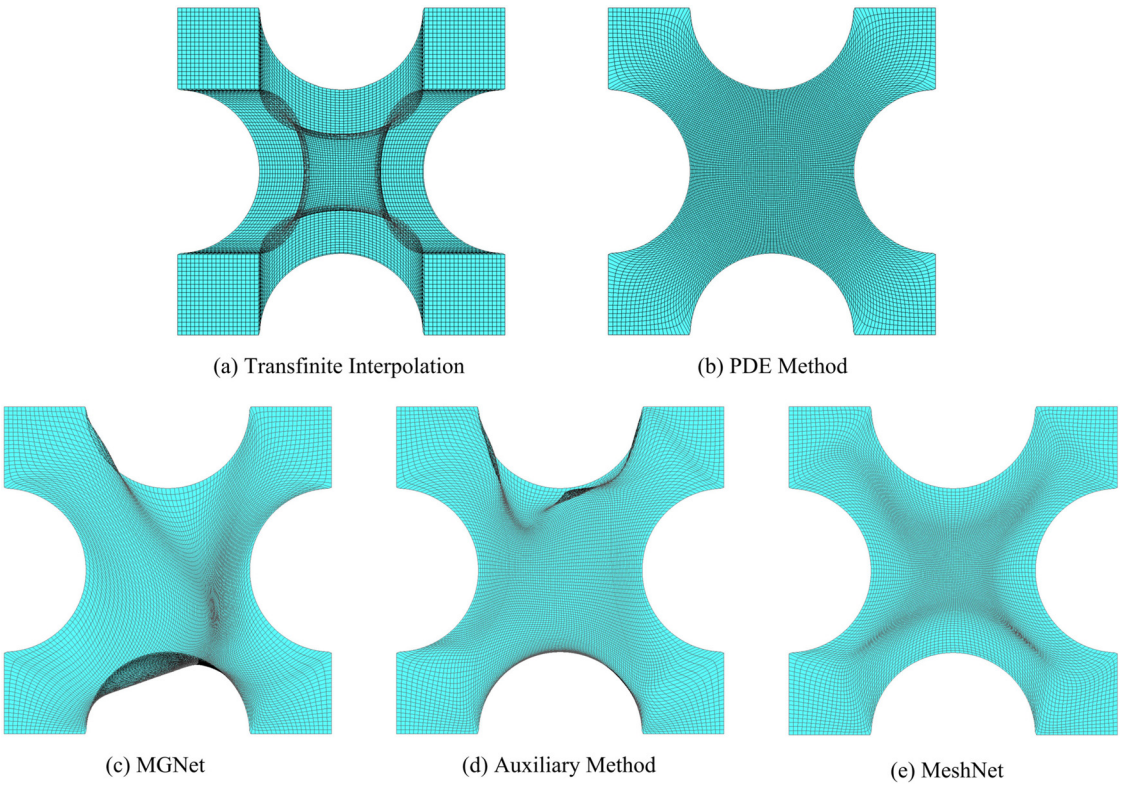

FIG. 5. The meshing results of different methods on geometric case 3.

Table III lists the mesh quality reports of the generated meshes. Notice that compared to the transfinite interpolation method, the LR_x (LR_y) of MeshNet is dropped from 5.323 (5.323) to 1.408 (1.439) but with little changes in $Max.A_{avg}$ (from 123.405 to 122.908). MGNet produces deformed cells in the region near the bottom boundary, which seriously affects the validity of the mesh. Similar to MGNet, the Auxiliary method outputs a slightly distorted mesh on geometric case 2. Because of distortions, both these two methods produce poorer quality results than MeshNet. For example, the aspect ratios of MGNet and Auxiliary methods are 1.765 and 1.545, respectively, while MeshNet is 1.083. Compared with the PDE method, the results of MeshNet on AR_{avg}, S_x, S_y, and *Skew* do not change much. MeshNet outperforms the PDE method on criteria such as LR_x, LR_y, and *Aspect* but is slightly worse than PDE on $Max.A_{avg}$ and *Skew*.

Figure 6 shows the meshes generated on the geometric case 4 using different mesh generation methods, and Table IV summarizes the quality reports for these meshes correspondingly. As can be seen, for geometries with complex boundary curves, the mesh lines are prone to be squeezed to the concave boundaries [see results in Figs. 6(a), 6(c), and 6(d)]. In contrast, both the traditional PDE method and the proposed MeshNet enable correct meshing results by the solution of the governing partial differential equation: a minimum overlapping between mesh lines is obviously ensured for geometric case 4. As expected, these two methods yield better quality evaluation results than the algebraic method using transfinite interpolation and the existing neural network-based generation methods. The *Skew* results of the PDE method and MeshNet are 0.289 and 0.211, respectively, while the other methods are above 0.3. They also perform better in AR_{avg} with results of 1.06 and 1.07, respectively.

TABLE III. Quality statistics of different mesh generation methods on geometric case 3.

Case 3	AR_{avg}	LR_x	LR_y	*Aspect*	S_x	S_y	$Max.A_{avg}$	*Skew*
TFI	1.701	5.323	5.323	40.443	0.975	0.975	123.405	0.372
PDE method	1.038	1.904	1.904	4.341	0.993	0.993	120.295	0.340
MGNet	1.765	1.418	4.533	60.203	0.987	0.987	140.226	0.560
Auxiliary method	1.545	1.514	3.718	14.255	0.988	0.986	127.574	0.420
MeshNet	1.083	1.408	1.439	3.158	0.989	0.990	122.908	0.369

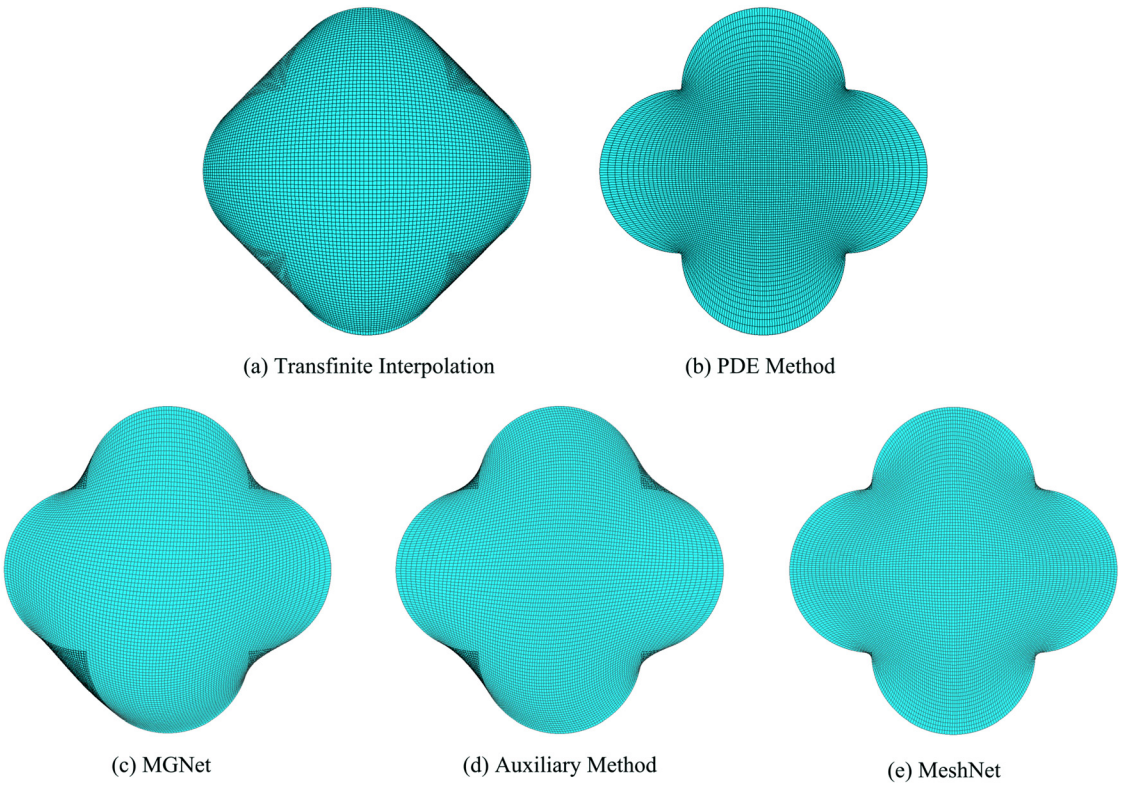

FIG. 6. The meshing results of different methods on geometric case 4.

However, they do not score as well as the existing neural network methods with respect to length ratio criteria (LR_x and LR_y). This is because the mesh points generated by NN-based generators are wrongly pushed to geometric boundaries, which leads to the illusion of a "uniform" transition of the mesh points, but, in fact, more distortions are observed. As for the smoothness criteria (S_x and S_y), little variation is observed in all methods, while higher $Max.A_{Avg}$ occurs in transfinite interpolation and MGNet due to their more tangling meshes.

Here, we present the meshing results on geometric case 5. Figure 7 shows the closeup views of the meshes generated by five different mesh generation methods. In general, a non-overlapping distribution is visible for all meshes, which indicates the minimal quality necessary to undertake a computational analysis. However, these generated meshes differ in terms of mesh cell transitions. Ideally, the change of the mesh element in the physical space is required to be a gradual transition (smooth transition) rather than a sudden transition. This means the mesh spacing should not be allowed to change too quickly. Uneven spacing can lead to the singularities of the reconstructed polynomial distribution, which affects the convergence and the accuracy of numerical results. For transfinite interpolation, since the mesh spacing in the circular arc region is smaller than in the regions on both sides of the arc, the resulting mesh suffers from severe smoothness problems. Although MGNet and Auxiliary methods can alleviate the uneven transition, they are limited by their learning capability and still cause uneven mesh transitions near the bottom boundary. In contrast, MeshNet and PDE methods produce the best smooth meshes, but the latter requires expensive meshing overhead due to the cumbersome numerical iterations.

TABLE IV. Quality statistics of different mesh generation methods on geometric case 4.

Case 4	AR_{avg}	LR_x	LR_y	*Aspect*	S_x	S_y	$Max.A_{avg}$	*Skew*
TFI	1.554	1.883	1.883	1.568	0.994	0.994	122.913	0.366
PDE method	1.060	1.458	1.458	3.126	0.995	0.995	108.864	0.211
MGNet	1.583	1.137	1.293	3.729	0.994	0.994	122.429	0.361
Auxiliary method	1.193	1.116	1.153	2.951	0.995	0.995	117.516	0.306
MeshNet	1.072	1.204	1.180	2.061	0.992	0.992	115.998	0.289

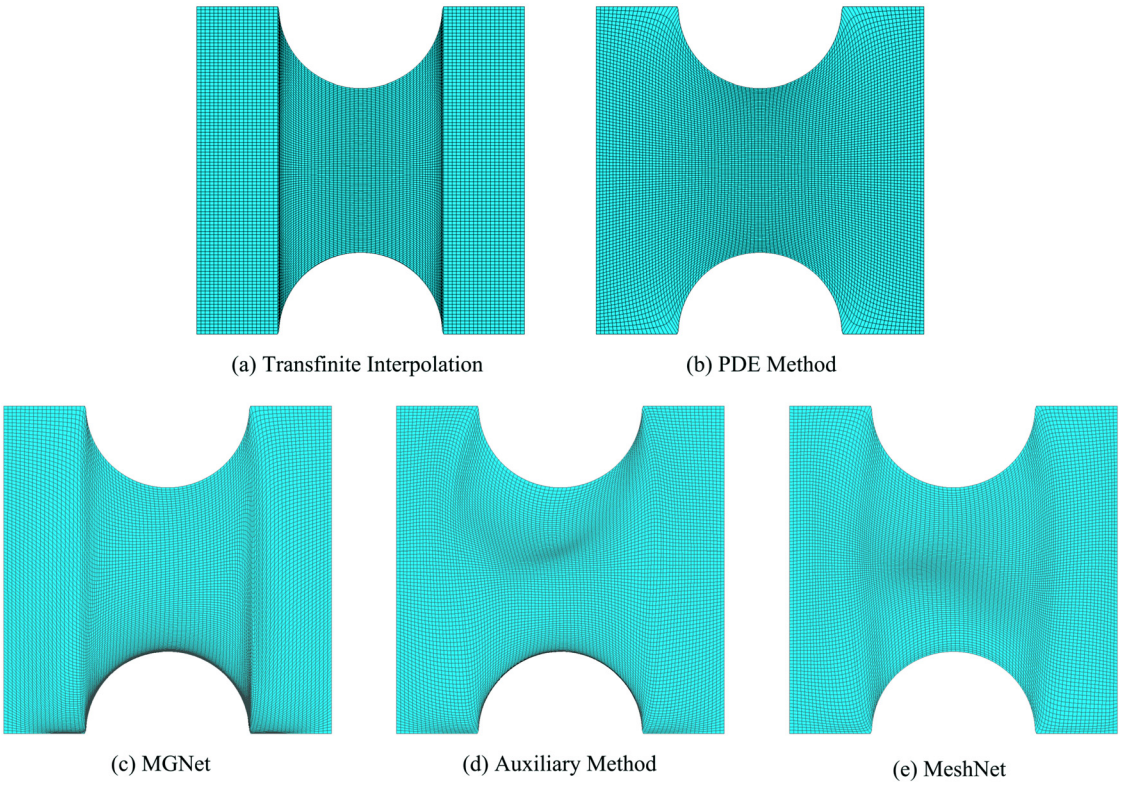

FIG. 7. The meshing results of different methods on geometric case 5.

Mesh quality criteria are also used to perform a quality analysis for the generated meshes. In this analysis, meshes are evaluated using different constant values, and the details are given in Table V. The evaluation results again prove that the TFI has cell transition pathologies along the x- and y-coordinate directions. The average length ratio of the method is 4.165 in the x-direction and 48.885 in the y-direction. As for the other methods, the PDE method achieves the best length ratio ($LR_x = 1.262$) in the x-direction, while the best length ratio in the y-direction is obtained by MeshNet ($LR_y = 1.630$).

In the following, we present the meshing results on geometric case 6. Figure 8 shows the final meshes using different methods, and Table VI presents the mesh quality report. Obviously, the traditional TFI method fails to generate an acceptable mesh without mesh distortion. We can also see that the MGNet leads to the deformation of the mesh near the boundary, which suggests the introduction of an observation constraint term to alleviate this problem. Although the Auxiliary method improves mesh smoothness compared to the MGNet method (LR_y decreases from 25.242 to 1.717, and *Aspect* decreases from 8.336 to 2.022), mesh deformation still remains in the domain [see Fig. 8(d)]. In the case of the PDE method, the mesh quality is greatly improved in smoothness at the cost of meshing overhead. The AR_{avg}, $Max.A_{avg}$, and *Skew* of the mesh generated by this method are 1.032, 100.144, and 0.114, respectively. However, it is clear that MeshNet using a composite loss function and improved network structure is able to produce comparable mesh. It scores 1.060, 101.013, and 0.124 on AR_{avg}, $Max.A_{avg}$, and *Skew* criterion, respectively.

It is well known that in most numerical applications, one has to minimize the deviation to orthogonality for preserving the numerical

TABLE V. Quality statistics of different mesh generation methods on geometric case 5.

Case 5	AR_{avg}	LR_x	LR_y	*Aspect*	S_x	S_y	$Max.A_{avg}$	*Skew*
TFI	1.312	4.165	48.885	124.374	0.983	1.000	108.192	0.202
PDE method	1.025	1.262	1.996	4.372	0.996	0.998	104.295	0.160
MGNet	1.176	2.811	2.930	120.447	0.978	0.997	121.562	0.351
Auxiliary method	1.218	1.362	4.481	318.354	0.988	0.996	111.849	0.244
MeshNet	1.058	1.339	1.630	5.104	0.990	0.998	105.749	0.176

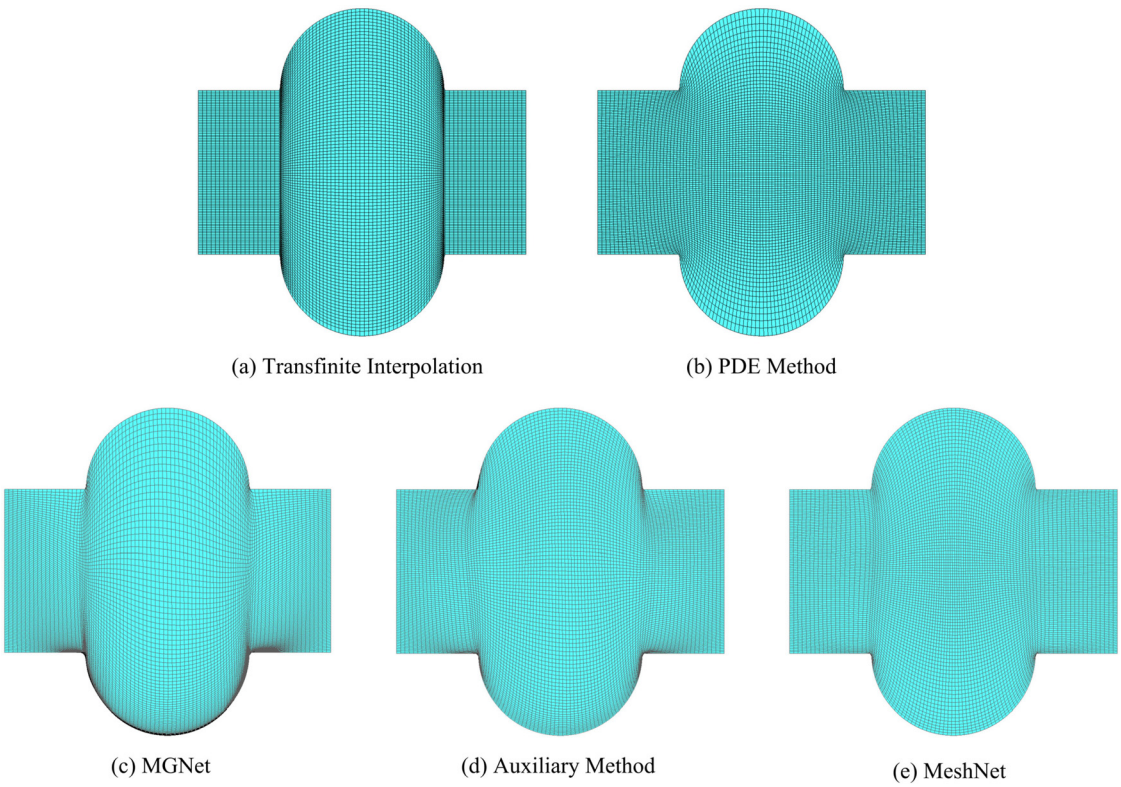

FIG. 8. The meshing results of different methods on geometric case 6.

scheme accuracy on a structured mesh. Mesh orthogonality means that the lines of the underlying mesh should be as orthogonal as possible. For example, the included angle of mesh cells should be close to 90 deg, or mesh lines should be normal to the geometric boundaries for the numerical scheme to be accurate. To this end, we use a circle as a geometric case to evaluate the orthogonal meshing ability of different methods. The visualization and statistic results are shown in Fig. 9 and Table VII, respectively. Mesh orthogonality can be indicated by the maximum included angle and the equiangle skewness criteria. The results indicate that, for the TFI method, the generated mesh is poor-orthogonal. For the PDE method, the maximum included angle is 102.678, and equiangle skewness is 0.141. For MGNet, the maximum deviation to orthogonality is driven by $Max.A_{avg}$ 113.328 and *Skew* 0.258 in this case. However, note that the proposed MeshNet improves mesh orthogonality greatly with the help of the well-designed loss function and network structure: $Max.A_{avg}$ is decreased from 107.403 in the Auxiliary method to 103.688 in MeshNet, while *Skew* is decreased from 0.193 to 0.159.

At last, the proposed method is challenged by a geometric case with irregular boundaries. As shown in Fig. 10, there is a discontinuous corner in the physical domain, which will make it difficult for quality mesh generation. We can see from the visualization results in Fig. 10(a) that the TFI tends to produce a severely skewed mesh at the corner. MGNet employs a neural network to learn the underlying mapping from the computational domain to the physical domain. The resulting mesh quality is slightly improved in smoothness compared with the algebraic method. However, the above-generated meshes still pose a problem of very poor distribution, and most numerical schemes

TABLE VI. Quality statistics of different mesh generation methods on geometric case 6.

Case 6	AR_{avg}	LR_x	LR_y	*Aspect*	S_x	S_y	$Max.A_{avg}$	*Skew*
TFI	1.256	4.165	1.497	1.815	0.983	1.000	108.192	0.202
PDE method	1.032	1.341	1.349	1.749	0.995	0.998	100.144	0.114
MGNet	1.475	1.579	25.242	8.336	0.985	0.994	123.080	0.368
Auxiliary method	1.221	1.529	1.717	2.022	0.988	0.998	107.860	0.200
MeshNet	1.060	1.467	1.521	1.855	0.990	0.998	101.013	0.124

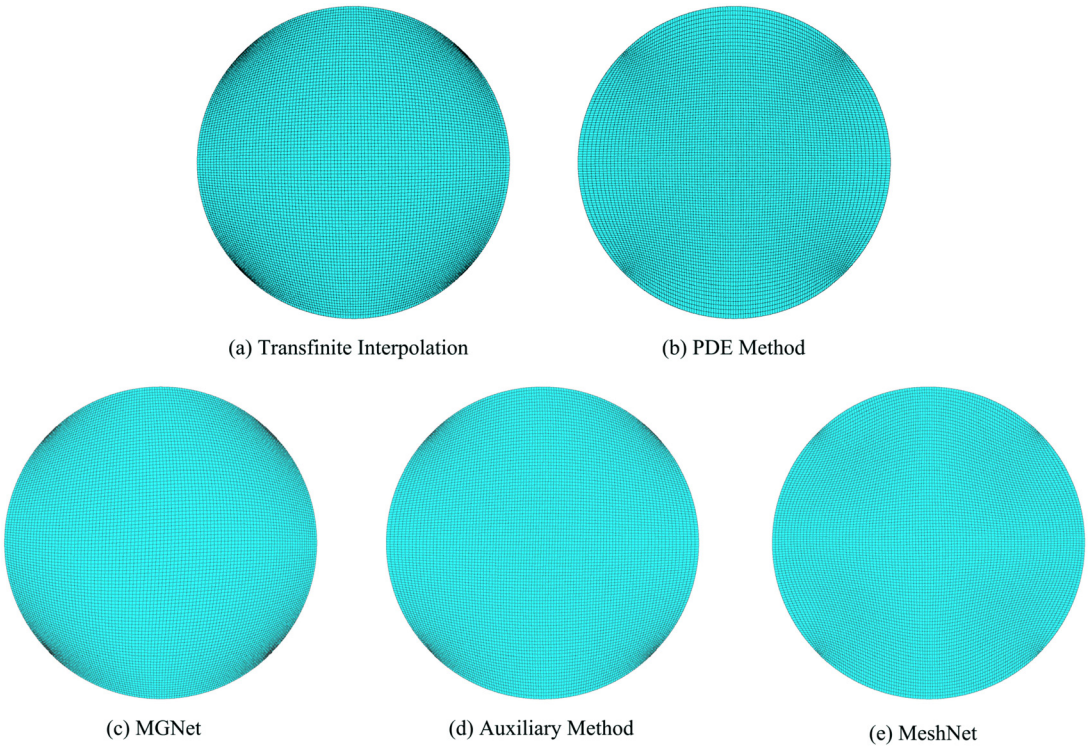

FIG. 9. The meshing results of different methods on geometric case 7.

on structured meshes produce high local errors in these poor distributed regions. With the auxiliary line strategy enforced, the Auxiliary method outputs a nearly smooth mesh [see Fig. 10(d)], but it is clear that MeshNet produces a better mesh in both mesh orthogonality and smoothness.

As previously, we used six criteria to comprehensively study the visualization results. The statistics results in Table VIII show that the MeshNet system is capable of producing acceptable meshes, where the desirable mesh transformation is jointly constrained by the governing equation, boundary, and observation terms. It achieves the best quality results on the LR_y (1.488) and *Aspect* (3.102) criteria. Meanwhile, the structured mesh is derived from the feed-forward predictions of the trained neural network and is, therefore, efficient. We can also find that for neural network-based generators, it is necessary to integrate *a priori* knowledge directly into the generation algorithm in order to make further progress on improving mesh quality. In particular, observation data selection has a substantial impact on the training quality and structured mesh quality, and a refined selection strategy has to take this into account.

C. Loss convergence

We now analyze the loss convergence of the proposed method, MeshNet. Unlike traditional algebraic or PDE methods, the main principle of the neural network-based generators is to learn the high-quality meshing rules and generate valid meshes through neural network training. These generators exploit the nonlinear learning ability of neurons in high-dimensional space to approximate the mapping from the computational domain to the physical domain. To validate the MeshNet design, we compare the convergence of MeshNet with

TABLE VII. Quality statistics of different mesh generation methods on geometric case 7.

Case 7	AR_{avg}	LR_x	LR_y	*Aspect*	S_x	S_y	$Max.A_{avg}$	*Skew*
TFI	1.033	1.328	1.328	1.026	0.998	0.998	110.176	0.224
PDE method	1.015	1.087	1.087	1.253	0.998	0.998	102.678	0.141
MGNet	1.040	1.165	1.151	1.060	0.997	0.997	113.238	0.258
Auxiliary method	1.021	1.115	1.088	1.140	0.998	0.998	107.403	0.193
MeshNet	1.010	1.057	1.043	1.219	0.996	0.996	103.688	0.159

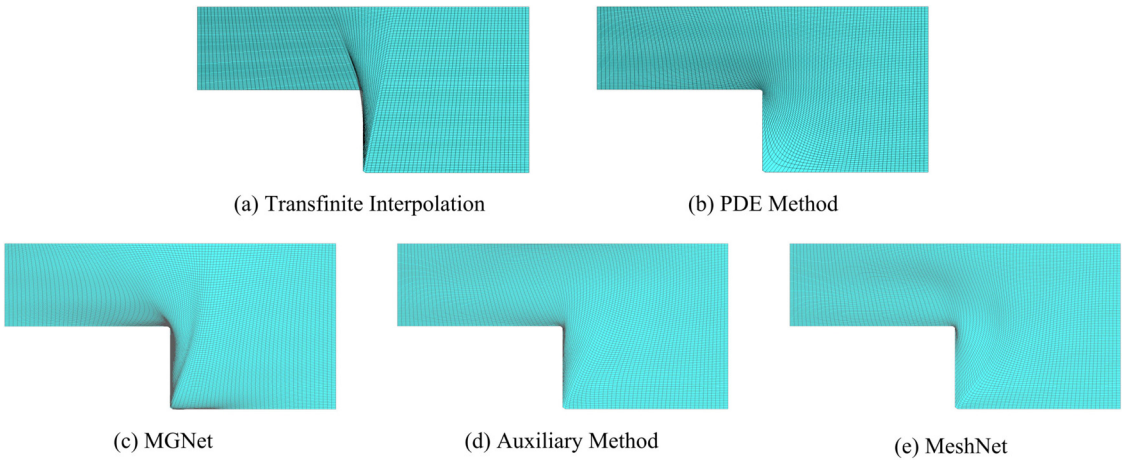

FIG. 10. The meshing results of different methods on geometric case 8.

two existing generators: the MGNet and Auxiliary methods. Figure 11 shows a detail of the loss convergence on case 5, while Fig. 12 depicts the variation curves of the loss value on case 7. In both figures, the green and red curves are generated using MGNet and Auxiliary methods, respectively, while the blue curve is obtained by MeshNet. To filter out the oscillations in the epoch series for the loss values, we employ a 1D Gaussian filter along the corresponding epoch series.

Here, we evaluate the efficiency of our proposed method by comparing the minimum loss value and the training iterations (epochs) on each loss term. As can be seen from the loss curves in Figs. 11 and 12, MGNet requires a lot of training overhead and effort to converge in two test cases. Taking geometric case 5 as an example, MGNet yields a loss value of 8.82×10^{-03} after 13 000 epochs, where the value of the governing equation residual is 6.54×10^{-05}, and the boundary term is 8.75×10^{-03}. With the auxiliary line strategy enforced, the Auxiliary method achieves smaller loss values. In case 7, the residual value converges to 5.86×10^{-05} after 9000 epochs. Similar trends are observed for the other loss terms. For example, the final loss value for the boundary term is 4.37×10^{-02} and 6.27×10^{-04} for the data term. However, it is clear that the proposed method outperforms existing NN-based generators in terms of training convergence, as it achieves the minimum convergence error. It rapidly converges and yields a total loss value of 3.57×10^{-03} on geometric case 5 and 3.02×10^{-02} on case 7. Finally, it should be reminded that minimization of the loss value is only a necessary condition to obtain an acceptable meshing result, but the reciprocal is not sufficient. This is because the loss function is a composite objective function. There exists the possibility of an invalid discretization due to unbalanced interactions between different constraint terms.

D. Hyperparameter settings

It is well known that different choices of hyperparameters have a significant impact on neural network training and prediction. In order to establish a training methodology, we explore the effects of various hyperparameter settings, including coarse mesh sizes, penalty weights, learning rates, and batch sizes, on the meshing performance of MeshNet.

1. Coarse mesh size

One of the most crucial hyperparameters for MeshNet is the coarse mesh size (N_d) associated with the observation term. This hyperparameter determines the granularity of the *a priori* data fed into the MeshNet and affects the data preprocessing overhead. We test the mesh performance of MeshNet in the N_d ranging from 5^2 to 25^2. We also import the generated meshes into the mesh preprocessing software and measure their quality using quality evaluation views. This procedure is performed quantitatively by employing the equiangle skewness criterion to evaluate the quality based on the degree of conformity and to highlight degenerate elements. The evaluation views

TABLE VIII. Quality statistics of different mesh generation methods on geometric case 8.

Case 8	AR_{avg}	LR_x	LR_y	*Aspect*	S_x	S_y	$Max.A_{avg}$	*Skew*
TFI	1.209	1.387	1.237	3.137	0.995	0.999	106.842	0.187
PDE method	1.023	1.387	2.251	3.123	0.997	0.999	103.199	0.147
MGNet	1.190	1.406	4.491	4.989	0.992	0.996	113.861	0.266
Auxiliary method	1.045	1.387	1.322	3.195	0.995	0.998	104.827	0.165
MeshNet	1.040	1.445	1.488	3.102	0.996	0.998	104.472	0.162

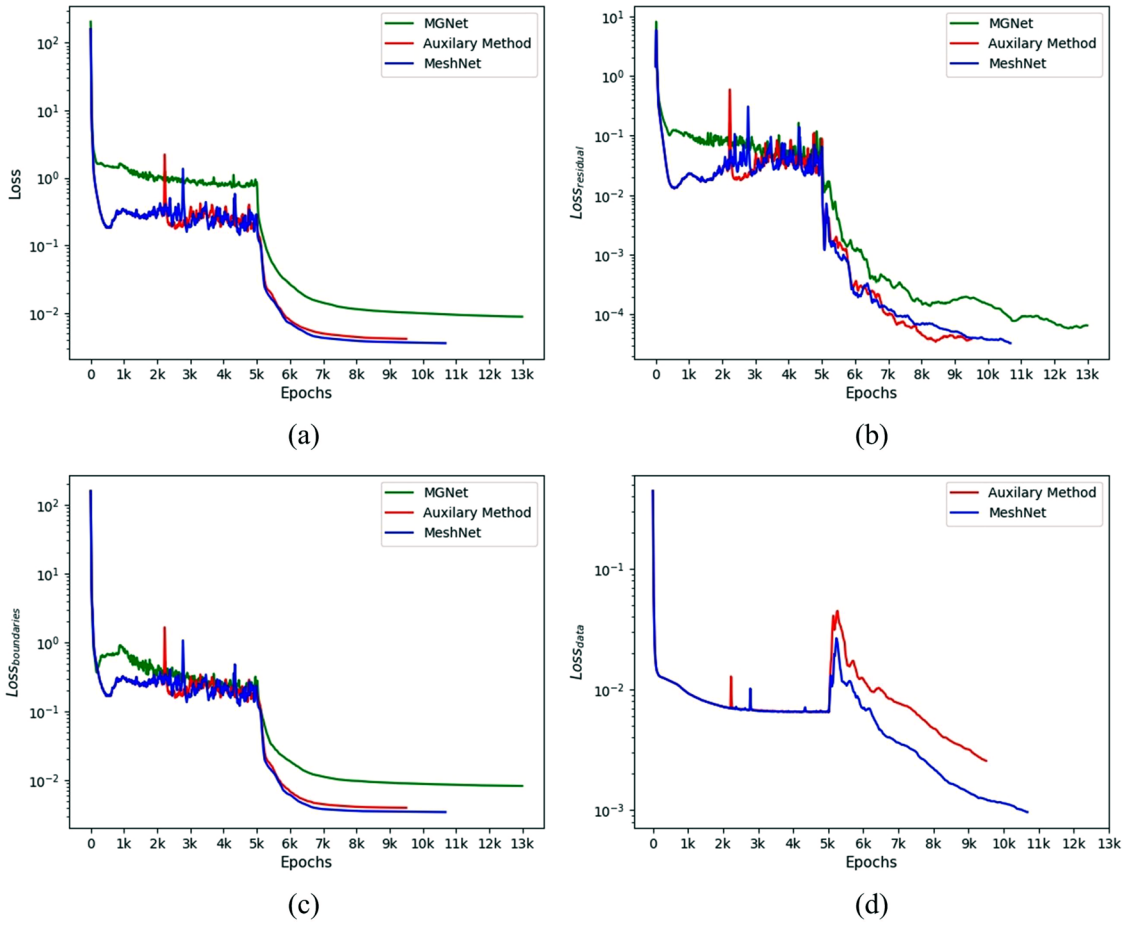

FIG. 11. A comparison of the convergence of different neural network-based methods on geometric case 5.

and statistics results of the final meshes on geometric case 2 are displayed in Fig. 13 and Table IX, respectively.

As expected, the meshes obtained using larger coarse mesh sizes have much better performance in mesh quality. This proves that MeshNet might have trouble converging, if the network is trained with insufficient observation points. When $N_d < 10^2$, MeshNet tends to produce meshes with severe mesh distortion and overlap [see Figs. 13(a) and 13(b)]. The *Skew* values of these generated meshes are greater than 0.3. When $N_d > 15^2$, MeshNet is able to produce high-quality meshes with $Skew < 0.26$.

2. Static penalty weight β

During the training, the static penalty weight β is employed to control and balance the contribution of different terms in the composite loss function. To this end, we conduct experiments on geometric case 3 to investigate the meshing performance of MeshNet when β values range from 0.05 to 500.

Figure 14 shows the quality evaluation views of the generated meshes, and Table X lists the trends of the equiangle skewness criterion with β increasing. It can be seen that small β might prevent convergence, while overly large β may get stuck in local minima, thus providing suboptimal solutions. For example, MeshNet with $\beta = 0.05$ produces a less smooth mesh than without the static weight ($\beta = 1$). However, we can observe that choosing a proper β value still has a positive effect on the MeshNet training. In general, the smoothness of the generated mesh can be promoted when $\beta > 1$. As shown in Fig. 14(d), MeshNet with $\beta = 10$ achieves the best meshing performance ($Skew = 0.368$). This implies that an optimal value of β could exist to balance the interplay between different terms in the loss function and accelerate convergence.

3. Learning rate and batch size

The learning rate is one of the most important hyperparameters in the gradient descent learning method. During the Adam-based training, this hyperparameter is the only sensitive value to control the gradient step size and, therefore, requires some adjustments. In practice, eight values in the range $[3 \times 10^{-5}, 1 \times 10^{-1}]$ are tried. As shown in Fig. 15(a), MeshNet achieves the best meshing performance for the

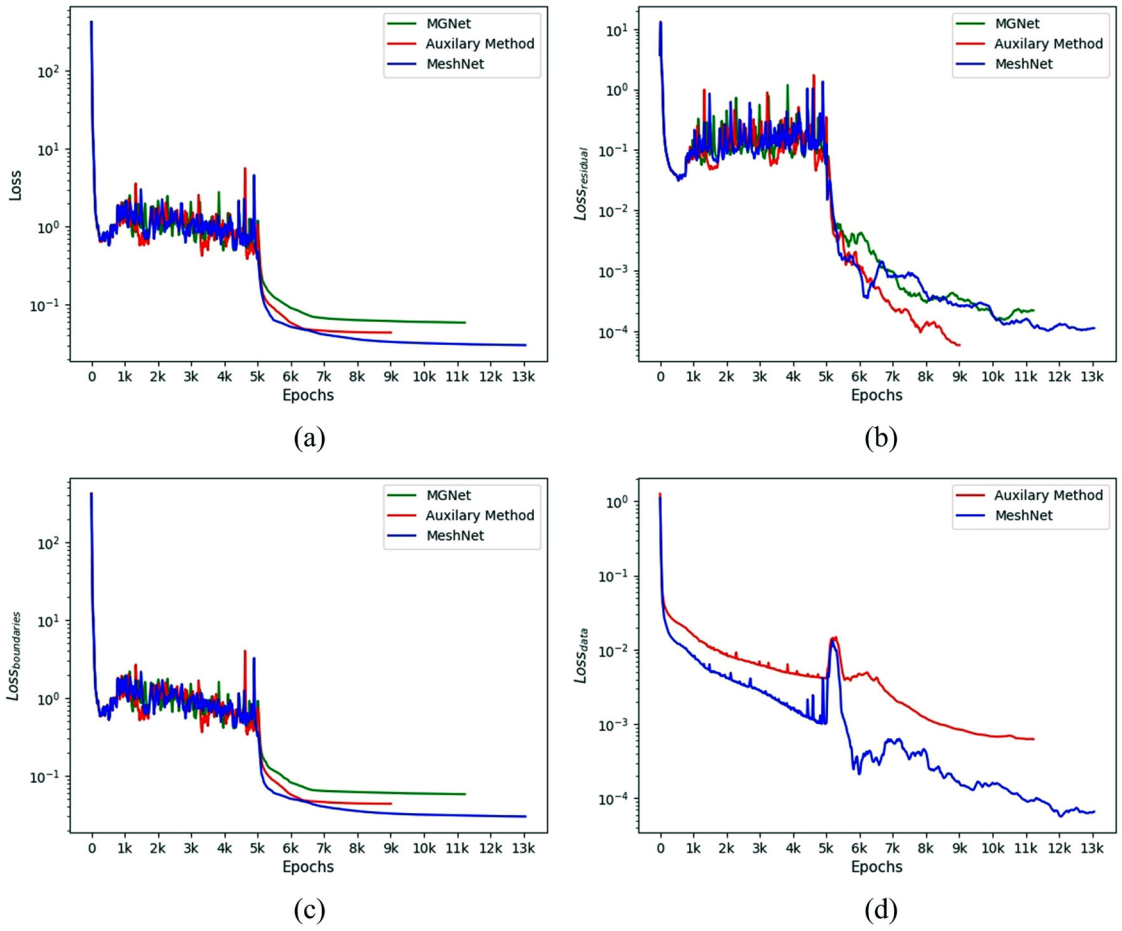

FIG. 12. A comparison of the convergence of different neural network-based methods on geometric case 7.

learning rate is 3×10^{-3}. We also include performance when MeshNet is trained on different batch sizes. Figure 16 shows the variation of the loss function with different values of batch size. Due to the memory limitations of the computing platform, we only give the results for batch sizes smaller than 2048. The experimental results demonstrate that a relatively large batch size is required to ensure the desired prediction accuracy, and the training error is minimized when the batch size is 128.

4. *Dynamic penalty weights* α

In addition, we find that assigning dynamic weights to the boundary terms, i.e., updating α prior to beginning each training epoch, helps to stabilize the results. Previous studies have demonstrated that a different range of values in loss terms is not good for the learning process. Loss terms with relatively larger values are prone to dominate the optimization direction while other terms vanish. It indicates that the serious mesh distortions and overlapping in geometrically complex cases may be caused by the imbalanced contribution of governing equation terms and boundary terms.

According to Eq. (6), the values of α depend on the two components of the composite loss function: MSE_g and MSE_b. In our work, we initialize their values by 10 and update them every 10 epochs. Figure 17 shows the trend curves of the two boundary weights with increasing epochs. Experimental results demonstrate that the introduced α can be used as a dynamic penalty strategy to automatically adjust the optimization direction, thus overcoming the imbalanced interplay between different terms. These dynamic weights can also improve the robustness of the proposed MeshNet, especially in scenarios where the training iterations are limited, and allow MeshNet to better capture the potential meshing features.

E. Meshing overhead

Here, we compare the meshing overhead from MeshNet with those produced using traditional techniques to verify the engineering application of the trained MeshNet as a valid generator. Seven different mesh sizes (from 100^2 to $10\,000^2$) are used to test the effect of mesh density on the mesh generation systems. In Table XI, we list the performance of the different methods by a series of execution times.

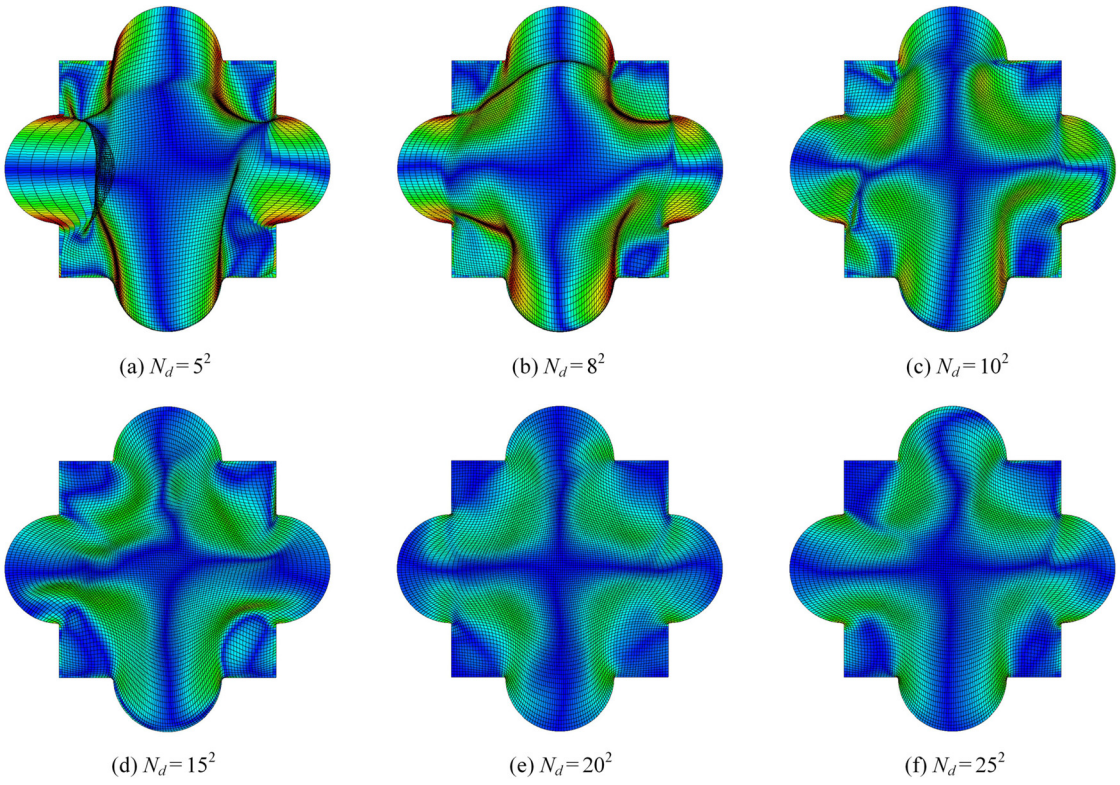

FIG. 13. Evaluation views of MeshNet with different coarse mesh sizes on geometric case 2.

TABLE IX. A comparison of mesh quality under different coarse mesh sizes.

N_d	5^2	8^2	10^2	15^2	20^2	25^2	30^2
Skew	0.304	0.344	0.277	0.259	0.233	0.242	0.236

All meshing processes are run in *Python*, single threaded on the aforementioned Intel Xeon E5–2660 CPU. Due to the fluctuations in the performance of the underlying computing platform, we run each meshing ten times and record the average time as the final overhead.

In general, the meshing overhead increases with the increase in the mesh size. As can be seen, the generation using TFI shows a linear scaling with the size of the objective meshes. The generation overhead is approximately 0.34 s when the mesh size is 40 000. For meshes of up to 100×10^6 cells, the meshing overhead is less than 11 min. However, despite its efficiency, the algebraic interpolation method usually produces seriously skewed or distorted meshes at complex boundary curves. These poor-quality meshes tend to slow convergence or cause numerical difficulties in the analysis process.

As for the PDE method, we compute the overhead of a finite number of elliptic smoothing (20 iterations). The results show that the PDE method suffers from tedious computational time. The expensive numerical iterations of the elliptic PDE system are the main contributors to the overall runtime of the execution. The execution times range from less than 12 min for a mesh size of 2000^2 up to about 5 h for the largest test size of 100×10^6 quadrilaterals. It becomes evident that the execution time of the PDE method is much higher than that of the algebraic or neural network methods.

We also observed that MeshNet achieves a good balance between mesh quality and meshing overhead. It is able to generate meshes of comparable quality to the PDE method at a very cheap cost. Again, there exists a near-linear scaling of the generation overhead. Solely for small inputs, small deviations from the linear behavior are visible. This is, however, not related to the meshing overhead of MeshNet but due to the initialization overhead at the start of the feed-forward prediction. With the increasing size of the desired structured mesh, this initialization overhead is relatively negligible, showing better linear scaling. Moreover, due to the independency nature of the feed-forward prediction, MeshNet can be easily parallelized on modern computing systems without code refactoring, which enables much faster execution times.

Overall, we propose a novel structured mesh generation method, MeshNet, for fast and robust meshing. The key to the success of MeshNet is the neural unit that is capable of universal approximation. The method uses these neurons to automatically learn the potential transformation (mapping) between computational and physical domains from high-dimensional parameter spaces. After suitable training, MeshNet can be used as a black box to generate the desired structured meshes with different geometries and pre-designed sizes.

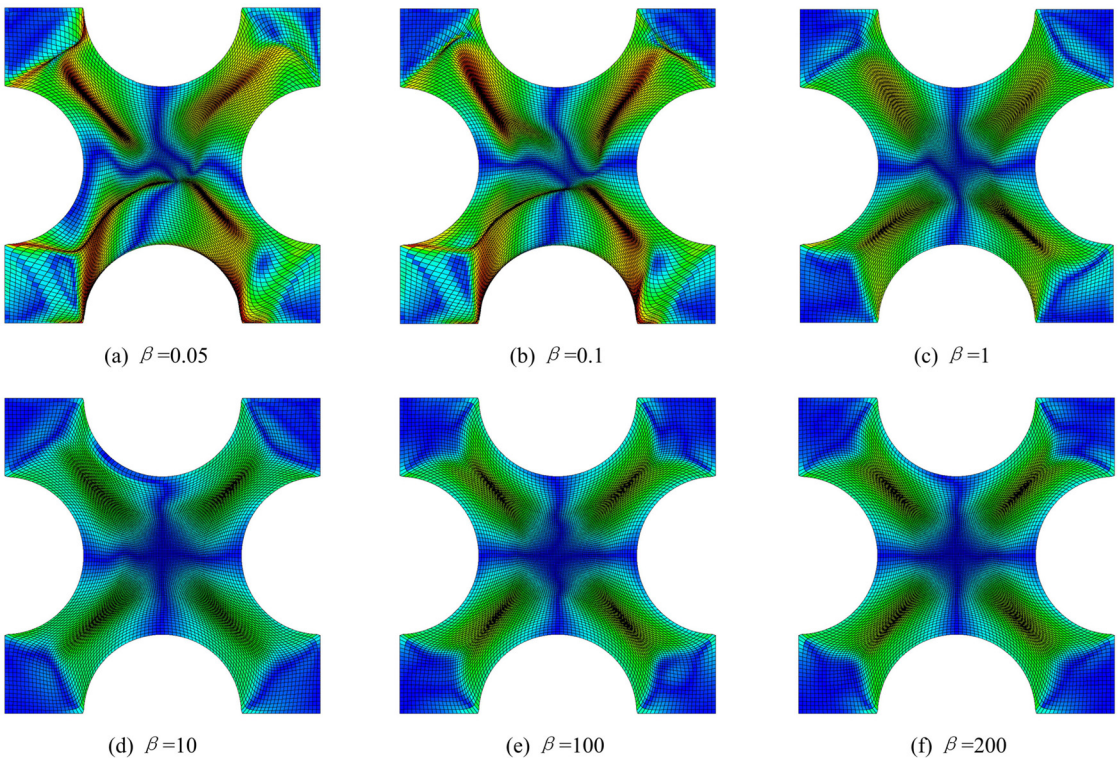

FIG. 14. Evaluation views of MeshNet with static penalty weights on geometric case 3.

TABLE X. A comparison of mesh quality under different static penalty weight β.

	0.05	0.1	1	10	100	200	500
Skew	0.478	0.465	0.420	0.368	0.393	0.392	0.403

Beyond its novelty, the efficiency of MeshNet has been demonstrated, wherein the meshing procedure can be performed rapidly using feed-forward neural prediction. Note that in the proposed methodology, the neural networks are used to learn the potential mapping rules and capture a series of mapping relations inherent in a specific geometry case. Therefore, once the neural networks are properly trained, the density of the output mesh can be changed by varying the input size values without re-training. The simplicity and computation speed of MeshNet make it possible to be used as a novel mesh generator in pre-processing units.

IV. CONCLUSION

Structured mesh generation is one of the fundamental and key issues in carrying out accurate numerical simulations of any physical

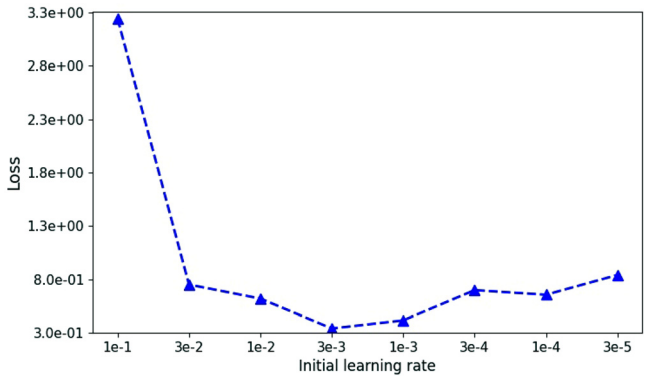

FIG. 15. A comparison of the minimum loss value for different learning rates.

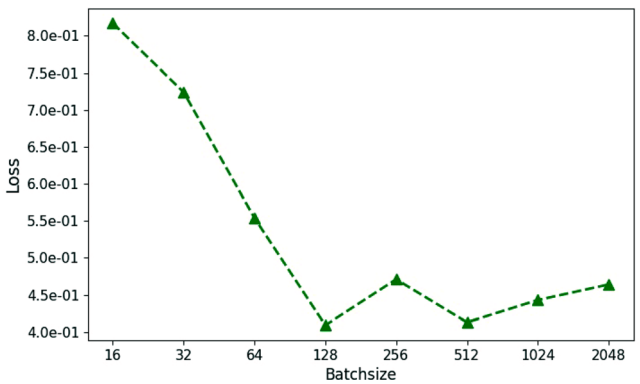

FIG. 16. A comparison of the minimum loss value for different batch sizes.

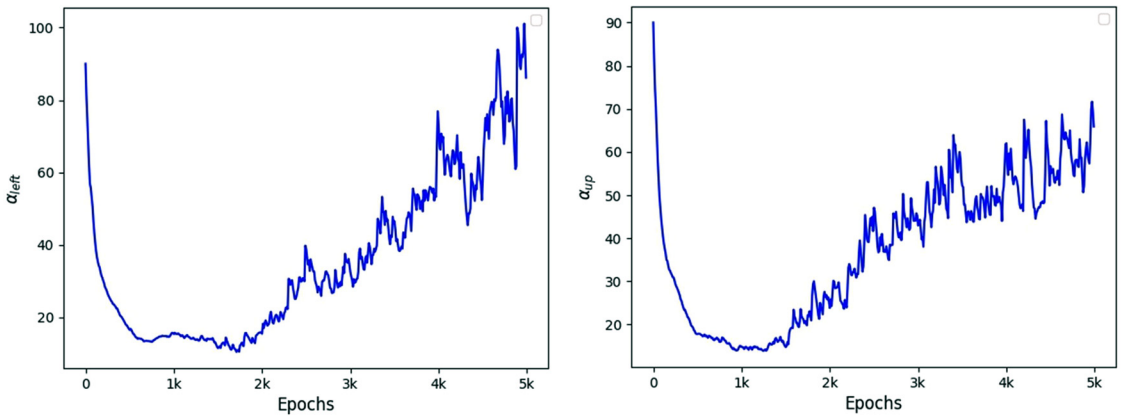

FIG. 17. The trend curves of four boundary weights with increasing epochs.

TABLE XI. A comparison of the meshing overhead for different mesh sizes.

	100^2	200^2	500^2	1000^2	2000^2	5000^2	$10\,000^2$
TFI	0.10	0.34	1.74	6.77	25.06	147.38	651.28
PDE method	2.44	6.96	43.83	175.13	712.82	4504.85	17 985.80
MeshNet	4.69	5.00	7.56	17.56	51.15	276.57	1160.75

model and structure. Existing structured mesh generation approaches suffer from a variety of robustness problems. For example, methods based on algebraic interpolation typically do not guarantee that the mesh cells are smooth and orthogonal everywhere, resulting in distorted lines and overlapping cells in the mesh. PDE methods pose difficulties in terms of meshing efficiency and computational overhead. On the other hand, both the learning capability and quality are often hard to control in previous neural network-based methods. In this paper, we develop an improved structured mesh generation MeshNet to overcome these problems. The proposed method consists first in fitting the input boundaries and then in generating observation data using an initial coarse mesh. Next, a well-designed network structure and composite loss function are employed to learn the high-quality meshing rules in the interior of the given boundaries. During the training, both attention mechanism and weighting strategies are introduced to accelerate convergence and disable the occurrence of degenerated mesh results. Finally, the meshing procedure is completed by feed-forward neural prediction, providing a fully automatic and efficient structured discretization.

To validate the proposed method, we compare the results obtained from eight distinct geometric cases with those generated by state-of-the-art neural network-based mesh generators as well as traditional algebraic and differential methods. Furthermore, we provide an in-depth analysis of various hyperparameter choices to establish an effective training methodology for neural network-based mesh generation tasks. Results serve to assess the robustness of the proposed method. Subsequently, we conduct experiments on the meshing overhead of MeshNet to verify its engineering application as a valid generator. Experimental results in all cases prove that MeshNet is capable of achieving robust structured mesh generation and showing comparable or superior meshing performance to the existing neural network-based and traditional methods.

Integrating deep neural networks into mesh generation tasks is a novel attempt. Despite the achievements mentioned earlier, more work should be performed in future works. First, additional research is needed to further improve the performance and efficiency of the methodology, including the development of enhanced neural network models, boundary fitting functions, differential operators, optimization mechanisms, and so on. Second, since the quality of the generated mesh appears to strongly depend on the governing equation, it will be helpful to duplicate the success of meshing studies in the geometry field by integrating more physical information, such as various smoothing conditions or enforced source terms, into the training. Third, in order to expand the applicability of the proposed method, our further work will consist in extending the method to three-dimensional configurations. Fourth, MeshNet is able to generate varisized meshes on a specific geometry without re-training. However, the current work does not impose any conditions for meshing across different boundary shapes. Therefore, the generalization ability of MeshNet can be considered for further improvement. For instance, it would be interesting to investigate the use of transfer learning or distillation models for meshing in similar geometries or slightly shape modification scenarios.

ACKNOWLEDGMENTS

This research work was supported in part by the National Key Research and Development Program of China (No. 2021YFB0300101).

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Xinhai Chen: Conceptualization (lead); Investigation (lead); Methodology (lead); Software (equal); Validation (lead); Writing – original draft (lead). **Jie Lu:** Funding acquisition (lead); Project administration (lead); Supervision (lead); Writing – review & editing (lead). **Qingyang Zhang:** Data curation (lead); Formal analysis (lead). **Jianpeng Liu:** Writing – review & editing (equal). **Qinglin Wang:** Resources (lead); Software (equal); Writing – review & editing (equal). **Liang Deng:** Software (supporting); Validation (supporting); Visualization (lead). **Yufei Pang:** Conceptualization (supporting); Formal analysis (supporting); Methodology (supporting); Project administration (supporting).

DATA AVAILABILITY

The data that support the findings of this study are available from the corresponding author upon reasonable request.

REFERENCES

[1]J. R. Chawner and N. J. Taylor, "Progress in geometry modeling and mesh generation toward the CFD vision 2030," AIAA Paper No. 2019-2945, 2019.
[2]C. Benoit and S. Peron, "Automatic structured mesh generation around two-dimensional bodies defined by polylines or PolyC1 curves," Comput. Fluids **61**, 64–76 (2012).
[3]Z. Fang, "A high-efficient hybrid physics-informed neural networks based on convolutional neural network," IEEE Trans. Neural Networks Learn. Syst. **33**, 5514–5526 (2022).
[4]P. Jin, Z. Zhang, I. G. Kevrekidis, and G. E. Karniadakis, "Learning Poisson systems and trajectories of autonomous systems via Poisson neural networks," IEEE Trans. Neural Networks Learn. Syst. (published online 2022).
[5]X. Chen, J. Liu, Y. Pang, J. Chen, L. Chi, and C. Gong, "Developing a new mesh quality evaluation method based on convolutional neural network," Eng. Appl. Comput. Fluid Mech. **14**, 391–400 (2020).
[6]D. Lowther and D. Dyck, "A density driven mesh generator guided by a neural network," IEEE Trans. Magn. **29**, 1927–1930 (1993).
[7]K. Xu and G. Chen, "Hexahedral mesh structure visualization and evaluation," IEEE Trans. Visualization Comput. Graph. **25**, 1173–1182 (2019).
[8]X. Chen, J. Liu, C. Gong, S. Li, Y. Pang, and B. Chen, "MVE-Net: An automatic 3-D structured mesh validity evaluation framework using deep neural networks," Comput.-Aided Des. **141**, 103104 (2021).
[9]J. Thompson, B. Soni, and N. Weatherill, *Handbook of Grid Generation* (CRC Press, 1998).
[10]G. Subramanian, A. Prasanth, and V. Raveendra, "An algorithm for two- and three-dimensional automatic structured mesh generation," Comput. Struct. **61**, 471–477 (1996).
[11]C. G. Armstrong, H. J. Fogg, C. M. Tierney, and T. T. Robinson, "Common themes in multi-block structured quad/hex mesh generation," Procedia Eng. **124**, 70–82 (2015).
[12]E. Ruiz-Girones and J. Sarrate, "Generation of structured meshes in multiply connected surfaces using submapping," Adv. Eng. Software **41**, 379–387 (2010).
[13]G. Jansen, R. Sohrabi, and S. A. Miller, "HULK—Simple and fast generation of structured hexahedral meshes for improved subsurface simulations," Comput. Geosci. **99**, 159–170 (2017).
[14]A. K. Singh, R. K. Gupta, M. Kumar, and K. Yadav, "Review of finite element mesh generation methods," AIP Conf. Proc. **2782**, 020095 (2023).
[15]T. J. Baker, "Mesh generation: Art or science?," Prog. Aerosp. Sci. **41**, 29–63 (2005).
[16]Y. Zhang, Y. Jia, and S. S. Wang, "An improved nearly-orthogonal structured mesh generation system with smoothness control functions," J. Comput. Phys. **231**, 5289–5305 (2012).
[17]X. Chen, C. Gong, Q. Wan, D. Liang, Y. Wan, Y. Liu, B. Chen, and J. Liu, "Transfer learning for deep neural network-based partial differential equations solving," Adv. Aerodyn. **3**, 36 (2021).
[18]X. Chen, R. Chen, Q. Wan, R. Xu, and J. Liu, "An improved data-free surrogate model for solving partial differential equations using deep neural networks," Sci. Rep. **11**, 19507 (2021).
[19]M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," J. Comput. Phys. **378**, 686–707 (2019).
[20]M. Raissi, A. Yazdani, and G. E. Karniadakis, "Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations," Science **367**, 1026–1030 (2020).
[21]H. Park, Y. Nam, J.-H. Kim, and J. Choo, "HyperTendril: Visual analytics for user-driven hyperparameter optimization of deep neural networks," IEEE Trans. Visualization Comput. Graph. **27**, 1407–1416 (2021).
[22]Y. Liu, W. Cao, W. Zhang, and Z. Xia, "Analysis on numerical stability and convergence of Reynolds averaged Navier–Stokes simulations from the perspective of coupling modes," Phys. Fluids **34**, 015120 (2022).
[23]L. Wang, Y. Fournier, J. F. Wald, and Y. Mesri, "A graph neural network-based framework to identify flow phenomena on unstructured meshes," Phys. Fluids **35**, 075149 (2023).
[24]P. Lu, N. Wang, X. Chang, L. Zhang, and Y. Wu, "An automatic isotropic/anisotropic hybrid grid generation technique for viscous flow simulations based on an artificial neural network," Chin. J. Aeronaut. **35**, 102 (2022).
[25]K. Huang, M. Krügener, A. Brown, F. Menhorn, H. Bungartz, and D. Hartmann, "Machine learning-based optimal mesh generation in computational fluid dynamics," arXiv:2102.12923 (2021).
[26]X. Lu and B. Tian, "Research on mesh generation in the finite element numerical analysis based on radial basis function neural network," in *2010 International Conference on Computer Application and System Modeling (ICCASM 2010)* (IEEE, 2010), Vol. 11, pp. 157–160.
[27]Z. Zhang, Y. Wang, P. Jimack, and H. Wang, "MeshingNet: A new mesh generation method based on deep learning," in *ICCS 2020: International Conference on Computational Science* (Springer, 2020), Vol. 12139, pp. 186–198.
[28]A. Papagiannopoulos, P. Clausen, and F. Avellan, "How to teach neural networks to mesh: Application on 2-D simplicial contours," Neural Networks **136**, 152–179 (2021).
[29]R. Chedid and N. Najjar, "Automatic finite-element mesh generation using artificial neural networks—Part I: Prediction of mesh density," IEEE Trans. Magn. **32**, 5173–5178 (1996).
[30]Ç. Ahmet and A. Ahmet, "Neural networks based mesh generation method in 2-D," in *EurAsia-ICT 2002: Information and Communication Technology*, edited by H. Shafazand and A. M. Tjoa (Springer, Berlin, Heidelberg, 2002), pp. 395–401.
[31]J. Bohn and M. Feischl, "Recurrent neural networks as optimal mesh refinement strategies," Comput. Math. Appl. **97**, 61–76 (2021).
[32]X. Chen, T. Li, Q. Wan, X. He, C. Gong, Y. Pang, and J. Liu, "MGNet: A novel differential mesh generation method based on unsupervised neural networks," Eng. Comput. **38**, 4409 (2022).
[33]X. Li, Y. Liu, and Z. Liu, "Physics-informed neural network based on a new adaptive gradient descent algorithm for solving partial differential equations of flow problems," Phys. Fluids **35**, 063608 (2023).
[34]Q. Hou, Z. Sun, L. He, and A. Karemat, "Orthogonal grid physics-informed neural networks: A neural network-based simulation tool for advection–diffusion–reaction problems," Phys. Fluids **34**, 077108 (2022).
[35]H. Eivazi, M. Tahani, P. Schlatter, and R. Vinuesa, "Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations," Phys. Fluids **34**, 075117 (2022).
[36]X. Chen, J. Yan, Z. Wang, C. Gong, and J. Liu, "An improved structured mesh generation method based on physics-informed neural networks," arXiv:2210.09546 (2022).
[37]F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, and G. Louppe, "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res. **12**, 2825–2830 (2012).
[38]D. You, C. F. Benitez-Quiroz, and A. M. Martinez, "Multiobjective optimization for model selection in kernel methods in regression," IEEE Trans. Neural Networks Learn. Syst. **25**, 1879–1893 (2014).

[39]J. Fu, H. Zheng, and T. Mei, "Look closer to see better: Recurrent attention convolutional neural network for fine-grained image recognition," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (IEEE, 2017), pp. 4476–4484.
[40]L. Xia, Y. Feng, Z. Guo, J. Ding, Y. Li, Y. Li, M. Ma, G. Gan, Y. Xu, J. Luo, Z. Shi, and Y. Guan, "MuLHiTA: A novel multiclass classification framework with multibranch LSTM and hierarchical temporal attention for early detection of mental stress," IEEE Trans. Neural Networks Learn. Syst. (published online 2022).
[41]J. Morales and J. Nocedal, "Remark on 'algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound constrained optimization,'" ACM Trans. Math. Software **38**, 1–4 (2011).
[42]M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: A system for large-scale machine learning," in *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI'16)* (USENIX Association, USA, 2016), pp. 265–283.